

PROJECT REPORT
PLANT DISEASE DETECTION USING DEEP LEARNING



Project Supervised By

Mr.Faisal Izhar

Submitted By

M Dilshad Hussain

BSF2102938

Mohsin Abid

BSF2102755

M Hamza Saeed

BSF2103037

BS IT

Session: 2021 – 2025

Department of Information Technology

University of Education, Lahore

Dera Ghazi Khan Campus

Plant Disease Detection Using Deep Learning

A project submitted in partial fulfillment of the
requirement for the award of the degree of BS
in Information Technology

BS in Information Technology: 2021 – 2025

**University of Education
Lahore,
DG KHAN CAMPUS**

“I hereby declare that I have read this project documentation and in my opinion this project is sufficient in term of scope and quality for the award of the degree of BS in Information Technology.”

Project Primary Supervisor

Name: Mr. Faisal Izhar

Designation: Lecturer

University of Education

Project Examiner

Name: Dr. Sajid Ali

Designation: Associate Prof.

University of Education

DECLARATION

I declare that project title entitled “**PLANT DISEASE DETECTION USING DEEP LEARNING**” is the result of our group own research and development except as cited in the reference. This project has not been accepted for any degree and is not concurrently submitted in candidate for any other degree. At any time if my statement is found to be incorrect even afterwards of BS Information Technology, the university has the right to withdraw my BS Information Technology.

Signature: _____

Name: Dilshad Hussain

Date: 15/05/2025

Signature: _____

Name: Mohsin Abid

Date: 15/05/2025

Signature: _____

Name: Hamza Saeed

Date: 15/05/2025

PLAGIARISM UNDERTAKEN

We solemnly declare that project work presented in this documentation entitles “PLANT DISEASE DETECTION USING DEEP LEARNING” is solely our work with no significant contribution from any other person. Small contribution wherever taken has been acknowledge and that complete project has been written by us.

We are aware of the HEC's and the University of Education, Lahore's zero-tolerance policy regarding plagiarism. As a result, as the author of the aforementioned project, we declare that neither the documentation for my project nor any materials used as references are properly cited.

We agree that the University reserves the right to revoke my BS degree if we are found to have committed any formal plagiarism in the above-mentioned project even after receiving it, and that the HEC and the University have the right to publish my name on the HEC/University Website alongside the names of students who have submitted plagiarized projects.

Signature: _____

Name: Dilshad Hussain

Date: 15/05/2025

Signature: _____

Name: Mohsin Abid

Date: 15/05/2025

Signature: _____

Name: Hamza Saeed

Date: 15/05/2025

CERTIFICATE OF APPROVAL

This is to ensure that the task work introduced in this documentation named, "PLANT DISEASE DETECTION USING DEEP LEARNING ", was directed by "Muhammad Dilshad Hussain, Mohsin Abid, Muhammad Hamza Saeed", under the oversight of "Mr. Faisal Izhar". This project has never been submitted for any degree elsewhere. The submission of this project to the "DG Khan campus, University of Education" fulfills a portion of the requirements for the BS in Information Technology degree.

Signature:_____	Signature:_____	Signature:_____
Name: Dilshad Hussain	Name: Mohsin Abid	Name: Hamza Saeed
Date: 15/05/2025	Date:15/05/2025	Date: 15/05/2025

Project Primary Supervisor
Name: MR. Faisal Izhar
Designation: Lecturer
University of Education

Project Examiner
Name: Dr. Sajid Ali
Designation: Associate Prof.
University of Education

OFFICE OF CONTROLLER OF EXAMINATION

No: _____

Date: _____

Muhammad Dilshad Hussain, Mohsin Abid, Muhammad Hamza Saeed, BS students in Information Technology at the University of Education DG Khan campus, have met all of the requirements for the award of a BS degree in information technology, as detailed below, and are therefore eligible for nomination.

BS INFORMATION TECHNOLOGY/

Cumulative Result

Credit Hours: _____

Registration No	Complete Name	Course work	Project	Total	Cumulative Grade Point Average (CGPA)

Project Title:

PLANT DISEASE DETECTION USING DEEP LEARNING

Name of Supervisor: Mr. Faisal Izhar

Signed by

Controller of Examination

ACKNOWLEDGEMENT

This report "A Task Documentation on plant disease Location Utilizing Profound learning" has been arranged as the course prerequisite for Conclusive year undertaking. The methods we used in our project to demonstrate how machines learn from data and behave like humans are the foundation of this report. The machine learning algorithms used to classify and detect the diseases of plants are described in this report.

We might want to says thanks to Mr.Faisal Izhar our venture boss who directed and energize us with the ideas of Man-made reasoning and AI. We would also like to thank Mr. Kashif Azhar and Mr. Umar Farooq for their ongoing guidance and assistance. We also thank Kaggle for its excellent Machine Learning course, which was very helpful in comprehending the fundamentals of various plant classification algorithms and methods.

We additionally might want to stretch out our genuine appreciation to our companions who gave us significant ideas on each part of task and assisted with forming this undertaking on clients' viewpoint. We would also like to thank the Department of Information Technology and its head, Mr. Kashif Azhar, for providing this wonderful opportunity. We expressed our gratitude to the university's DG Khan Campus principal, who made great efforts to provide all facilities for the 2021–2025 academic year.

Date: 15/05/2025

PREFACE

In today's world, computer systems are maturing to the point where they can function and act like humans and make decisions in the same way. All in all, we prepared our PC to pursue choices which kind of establish illness it is. Our primary objective is to detect the plant disease means which type of disease is and with this we also classify the disease of plant' types using different method of classification.

ABSTRACT

This is era of technology machines are use to know the disease of plants heavily used in a wide variety of marine ecology applications. However, in order to manage the enormous amount of data collected effectively, we must create technologies that can instantly identify species that have been photographed.

Python language use Anacoda Navigator(software) - library and dependencies use in it in we visual studio front end use in MAIN.PY buttons are used in MAIN.py Train plant disease file (libraries use in this are (tensorflow, matplotlib, pandas , seaborn) , (data processing (training image processing) , images data 70295 (38 classes) validation image preprocessing (after flirt images) images data 17572 (38 classes) after all it make the training set and convert it & show the training shape in x shape and y shape Builing Model we use keras library and include the DENSE, CONV2D, MAXPOOL2D, FLATTEN and the method we use for it thats sequential Builing convolution layer 1 size of images, and library use (DENSE, CONV2D, MAXPOOL2D, FLATTEN) use like a CSS file train all the procedure as four, five times. Compiling Model it show the accuracy and also show the summary and show the table (in that table it will show the LAYER(TYPE), OUTPUT SHAPE,PARAM) total params 7510,890 trainable param 7150890 Non-trainable params Model Training save the history , all the procedure run and run the code at 10 times, we can re-write 10 times , 20 times to run the model training, in this model training it shows the ACCURACY, LOSS, VOL-ACCURACY AND VAL-LOSS.

Model Evaulation how much training -loss & training-acc evaluate into training set and print the training accuracy & loss. Saving Model we will save the model and give the model name in "TRAINED_MODEL.KERAS" we will search the training_history.history in that searched we see the all accuracy of pictures,in that searched we see the all LOSS of pictures. in that searched we see the all VOL-accuracy of pictures,in that searched we see the all VOL-LOSS of pictures. after all doing all the procudure we will print the keys , the keys will be (ACCURACY ,VOL-ACCURACY ,LOSS VOL-LOSS).

ACCURACY Visulalization ephochs = {i for i in range(1,11)}, it will pick the history data and show the accuracy , colour, lable, loss see inthe GRAPH SOME OTHER METRICS FOR MODEL EVALUATION name of classess , there is no show that how many pictures are there in it. by using tf.keras all the dataset of all images (after filter the 17572 picture belonging to the 38 classess) , we use the class names="NONE", colour_mode="rgb",batch_size=1 image_size=(128,128), shuffle=false ,seed=none, subset=none after this it will predict all the categories.

from sklearn.metrics import confusion_metrics, classification report(it will inthe form of table of classification report) in this form it will show the classes (classes like APPLE,Grape,Corn) PRECISION, RECALL, FI-SCORE & SUPPORT at the last it will show all the accuracy,macro avg, weighted avg.

CONFUSSION METRIC VISUALIZATION we make a complete table , a complete metrix(we use the FIGURE (figsize(40.40), HEATMAP(size="10"),XLABEL(font-size="20"),YLABEL(font-size="20"),

TITLE(title="PLANT DISEASE PREDICITION CONFUSION METRIX" font-size="25"), SHOW) the data we use in training , similalry we will use the same data inthe test file first one is "TEST SET IMAGE PROCESING" second one "LOADING MODEL. LOADING MODEL the model we save inthe training that is KERAS , we use this in for loading "trained_plant_disease_model.keras"

model="sequential" in the model it will show the training data model that show the "Layer(type)", "Output shape" & "Params" total param ="22.532.672(85.96 MB)", trainable param:"7,510,890 (28.65 MB)" , non-trainable params:"0(0.00)", Optimizer params:"15,021,782 (57.30MB)"

VISUALISING & PERFORMING Prediction on single image it will test the single image and show the result. we code in this and show `imgshow(img)`, `tilte("TEST Image")`, `xtick([])`, `ytick([])`, `show()`

TESTING MODEL we will test the model and `print(predication)`, in result of index it show the only 2 , after preidiction it show the result and also show all the class name

Displying the disease prediction `model_prediction = class_name(result_index)`, `imgshow(img)`, `tilte("Disease name:(model_prediction)")`, `xtick([])`, `ytick([])`, `show()` and show the image & show the disease

we run the file as MAIN.PY(STREAMLIT library we used) for running we use the commadn "streamlit run main.py" after this it show us the front-end view as APP i also create HTML file name as INDEX.HTML(in that file we use CSS)

in the front-end view it show us 'HOW IT WORKS' (upload the image),(Analysis the image), (show the result along with disease) in the front-end view it show us 'WHY CHOOSE US' i(accuracy) ii(user-friendly),iii(fast& efficient) we have a button 'ABOUT US PAGE' (after click on this it show) traing"70,295 image" , TEST:"33 image ", Validation:"17,572 images" we have also a button 'Go to Disease Recognition page'(at the same time will select only three images) we select the images and click on button 'Prediciton' after prediction it show the table , in that table it show the 'image', 'Disease' & 'confidence %'

PROJECT BRIEF

Project Name:	PLANT DISEASE DETECTION USING DEEP LEARNING
Supervised by	Mr. Faisal Izhar
Date of starting	10-10-2024
Completion Date	01-05-2025
Operating System	Microsoft Window 10
System Used	HP Core i5
Source of language	Python
Platform:	Anacoda Navigator
GPU	Library and dependencies

TABLE OF CONTENTS

CHAPTER 01 Gathering & Analysing Information.....	01
1.1 Introduction.....	02
1.2 Motivation.....	03
1.3 Scope.....	03
1.4 Objective.....	04
Chapter 02 Background.....	05
2.1 What is Artificial Intelligence.....	06
2.1.1 Different Categories of Artificial Intelligence.....	06
2.2 Work of AI.....	06
2.3 Machine Learning.....	06
2.3.1 Artificial Intelligence Applications.....	08
2.3.2 How Machine Learning Works.....	08
2.3.3 Machine Learning Models fall into three primary categories.....	09
2.3.4 Common Machine Learning Algorithms.....	09
2.4 What is Deep Learning?.....	10
2.4.1 Rise of Deep Learning.....	11
2.4.2 Deep Learning in Action.....	11
2.4.3 Example of Deep Learning at Work.....	12
2.4.4 How Deep Learning Works.....	12
2.5 Difference Between Machine Learning and Deep Learning.....	13
2.5.1 Two main key points are used in both type of learning.....	13
2.6 Transfer Learning.....	14
2.7 Neural Network.....	16
2.7.1 Types of neural networks.....	17
2.8Convential Neural Network.....	17
2.8.1 Applications.....	20
2.8.2 Motivation Behind Convolution.....	20
2.9 Available Methodology.....	22
2.10 Research Work.....	23
2.11 Chosen Methodology.....	24
CHAPTER NO 03 DATA SET PREPARTION.....	25
3.1 Data Collection.....	26

3.2 Train Plant Disease.....	26
3.3 Work Plan.....	27
3.4 Builing Model.....	28
Chapter 04 PLANNING PROJECT.....	30
4.1 Introduction.....	31
4.2 Research Methodology.....	31
4.2.1 Model Evaulation.....	32
4.3 Saving Model.....	32
4.5 Block Diagram.....	33
CHAPTER 05 DEVELOPMENT.....	38
5.1 SOME OTHER METRICS FOR MODEL EVALUATION.....	39
5.2 CONFUSION METRIX VISUALIZATION.....	39
5.3 Accuracy Visulization & Model Training.....	41
5.4 LOADING MODEL.....	43
5.5 VISUALISING & PERFORMING Prediction on single image.....	45
5.6 TESTING MODEL.....	46
5.7 Displying the disease prediction	47
5.8 Summary of Model.....	48
5.9 Tensorflow.....	50
CHAPTER 06 USER INTERFACE.....	51
6.1 Plant Disease Recognition System.....	52
6.2 Final Detection Result.....	55
<i>CHAPTER N0 07 Conclusin & Future work.....</i>	58
Conclusion.....	59
Future Work.....	59

List of Figure

Figure 4.1 Methodology	24
Figure 4.1 Block Diagram.....	33
Figure 5.1 Accuracy visulization.....	41
Figure 5.2 Model Training.....	42
Figure 5.3 Summary of Model.....	48
Figure 6.1 Plant disease Recognize system Interface.....	54
Figure 6.2 Accuracy of Corn_(maize)___Common_rust and Potato__Early_blight.....	55
Figure 6.3 Accuracy Result of Grape_Black_rot and Peach_Bacterial_spot.....	56

CHAPTER 01

GATHERING AND ANALYZING INFORMATION

1.1 Introduction

Agriculture remains the backbone of many economies worldwide, providing food, raw materials, and employment to a significant portion of the population. However, one of the major challenges faced by the agricultural sector is the management and control of plant diseases. Plant diseases can cause substantial losses in crop yield and quality, severely impacting farmers' livelihoods and national food security. Traditionally, identifying plant diseases has relied heavily on manual inspection by farmers or agricultural experts. This approach is not only time-consuming but also prone to human error, especially when the symptoms of different diseases appear similar. In this context, technological advancements, particularly in the field of deep learning, have opened new doors for enhancing the accuracy, efficiency, and scalability of plant disease detection.

Deep learning, a subset of artificial intelligence (AI) that mimics the way humans gain knowledge, has shown tremendous promise in the area of image recognition and classification. By using deep learning models, particularly convolutional neural networks (CNNs), it is now possible to develop systems capable of identifying plant diseases from images of leaves with remarkable precision. These systems analyze thousands of features in an image—far beyond what the human eye can detect—and learn to associate specific patterns with specific diseases.

The main benefit of using deep learning for plant disease detection is the significant increase in **speed and accuracy**. A well-trained deep learning model can diagnose a plant disease in a matter of seconds, enabling quicker responses and early intervention, which are crucial for limiting the spread of the disease. Additionally, the system can work 24/7 without fatigue, ensuring consistent and reliable diagnosis at any time. This not only saves farmers time and resources but also reduces the dependency on agricultural experts, which can be scarce in remote or rural areas.

Moreover, deep learning models can continually improve over time. As more images are fed into the system, the model becomes better at distinguishing between different diseases, even when they have subtle differences. This adaptive learning capability ensures that the system stays effective as it encounters new variations of diseases or changes in environmental conditions. Such flexibility is a significant advantage over traditional static diagnostic methods.

In terms of **specifications**, a plant disease detection model typically consists of several key components. First, a large dataset of plant images, including healthy and diseased samples, is required. The popular PlantVillage dataset, for example, contains over 50,000 images of healthy and infected leaves across different crop species and diseases. These images are preprocessed—resized, normalized, and augmented—to prepare them for model training.

Next, the model architecture plays a vital role. Convolutional Neural Networks (CNNs) are commonly used because they are excellent at capturing spatial hierarchies in images. A typical CNN model for plant disease detection might include multiple convolutional layers for feature extraction, pooling layers for dimensionality reduction, and fully connected layers for final classification. Sometimes, transfer learning techniques are employed, where a model pre-trained on a large dataset like ImageNet is fine-tuned for plant disease detection, drastically reducing the required training time and increasing accuracy.

The training process involves feeding the model thousands of labeled images so that it can learn to differentiate between different disease types. During evaluation, the model's performance is assessed using metrics such as accuracy, precision, recall, and F1-score to ensure its reliability in real-world scenarios. Once trained and validated, the model can be deployed through various platforms, such as a mobile application, web interface, or integrated into farm management systems.

Regarding **usage**, this model has vast applications across various sectors of agriculture. Farmers can use it directly through mobile apps by taking a photo of an infected leaf and receiving immediate diagnosis along with suggested treatments. Agricultural extension workers can employ the system to assist multiple farmers quickly during field visits. Government agencies and NGOs can integrate the model into large-scale crop monitoring systems to predict disease outbreaks and plan preventive measures. Even researchers and agronomists can use the system to speed up the identification process in experimental studies.

Furthermore, by combining plant disease detection with geographic information systems (GIS) and Internet of Things (IoT) devices, it is possible to develop smart farming solutions that monitor plant health in real-time across vast agricultural areas. This could lead to more sustainable farming practices by optimizing the use of pesticides, improving crop yields, and ultimately supporting global food security initiatives.

In conclusion, the application of deep learning for plant disease detection represents a transformative step in modern agriculture. It addresses critical challenges in disease management by providing a fast, reliable, and scalable solution. As the technology matures, it will continue to offer more sophisticated and accessible tools to farmers worldwide, helping to ensure a healthier and more prosperous agricultural future.

1.2 Motivation

The main motivation behind this system is to learn and develop an effective plant disease detection model by applying knowledge from previous projects and integrating new ideas and techniques. The motivation for developing plant disease detection using deep learning stems from the urgent need to support farmers by identifying plant diseases quickly and accurately. Agriculture plays a vital role in the global economy, and crop diseases can lead to massive losses if not detected at the right time. Traditional methods of identifying plant diseases are often time-consuming, error-prone, and require expert knowledge. By utilizing powerful Python libraries and dependencies such as TensorFlow, Keras, OpenCV, and scikit-learn, we aim to build a highly efficient and automated system without the need for manual feature extraction or deep CNN model building from scratch.

This project focuses on leveraging pre-built deep learning models, datasets, and image processing tools to streamline the disease detection process. It enables faster development, easier maintenance, and ensures high accuracy with less complexity. Our goal is to assist farmers, agricultural researchers, and organizations by providing a reliable tool that supports early disease detection, promotes healthy crop production, and encourages the adoption of AI-driven solutions to enhance global food security and sustainable agriculture.

1.3 Scope

According to a recent research paper, each uses a variety of approaches to improve

accuracy. Despite the fact that their work yields positive outcomes, they do not employ any effective models that produce results that exceed 95%. For their classification system, we utilized this model. It gives us more than 90 % results.

1.4 Objective

The primary objective of this system is to learn about it and contribute to its development by utilizing knowledge gained from previously constructed project concepts and novel ideas from diverse minds. It is crucial to develop an automated system that can assist farmers in automating the classification of disease of plants rather than manually doing so. Deep learning-based object recognition offers a quick and precise way to forecast where an object will appear in an image. The object detector automatically learns the visual features needed for detection tasks using deep learning, a potent machine learning technique.

Chapter 02

BACKGROUND

2.1 What is Artificial Intelligence

Artificial intelligence (AI) refers to simulation human intelligence in machine that are programmed to think like human and act like human. The term may also be applied to any machine that exhibits traits associated with human mind such as learning and problem solving. Machine learn from previous data and perform like as human. Subset of artificial intelligence is machine learning that can learn from data and act like human. Deep learning also subset of machine learning based on artificial neural networks in which multiple layers of processing are used to extract progressively higher level features from data. Artificial intelligence (AI) is the term used to describe the simulation of human intelligence in machines that have been designed to think and behave like people. Any machine that demonstrates characteristics of the human mind, like as learning and problem-solving, may also be referred to by this phrase. Machines can mimic human performance by learning from prior data. Machine learning is a subset of artificial intelligence that can act like a human and learn from data. Deep learning is a subset of machine learning based on artificial neural networks in which successively more complex features are extracted from data using numerous layers of processing.

2.1.1 Different Categories of Artificial Intelligence

Weak Artificial Intelligence: Weak artificial intelligence (AI) also called narrow AI is a type of artificial intelligence that is limited to a specific or narrow area. Today we used weak AI because machine performs limited tasks

Strong Artificial Intelligence: Strong artificial intelligence (strong AI) is an artificial intelligence construct that has mental capabilities and functions that mimic the human brain. Future AI also called strong AI because it performs unlimited tasks and work like human brain e.g robotics.

2.2 Work of AI

Face detection, products recommendation, customer interests, disease classification, fake accounts classification on social media, banking loan eligibility checking, fraud detection in transactions, youtube video recommendations, AI based attendance system, AI based chatbots, fashion recommendation, robotics works in big companies and many other fields.

2.3 Machine Learning

Machine learning is a branch of artificial intelligence (AI) and computer science which focuses on the use of data and algorithms to imitate the way that humans learn, gradually improving its accuracy.

Over the last couple of decades, the technological advances in storage and processing power have enabled some innovative products based on machine learning, such as Netflix's recommendation engine and self-driving cars.

Machine learning is an important component of the growing field of data science. Through the use of statistical methods, algorithms are trained to make classifications or predictions, and to uncover key insights in data mining projects. These insights

subsequently drive decision making within applications and businesses, ideally impacting key growth metrics. As big data continues to expand and grow, the market demand for data scientists will increase. They will be required to help identify the most relevant business questions and the data to answer them. Machine learning algorithms are typically created using frameworks that accelerate solution development, such as Tensor Flow and PyTorch.

A subfield of artificial intelligence (AI) and computer science called machine learning focuses on using data and algorithms to simulate how people learn, progressively increasing the accuracy of the system. Machine learning-based products like Netflix's recommendation engine and self-driving cars have been made possible in recent years because to technical advancements in storage and processing capability. The rapidly expanding discipline of data science includes machine learning as a key element. Algorithms are taught using statistical techniques to produce classifications or predictions and to find important insights in data mining projects. The decisions made as a result of these insights influence key growth indicators in applications and enterprises, ideally. Data scientists will be more in demand as big data continues to develop and flourish. They will be expected to assist in determining the most pertinent business issues and the information needed to address them. The majority of the time, machine learning algorithms are developed utilizing accelerated solution development frameworks like Tensor Flow and PyTorch.

In general, predictions or classifications are made using machine learning algorithms. Your algorithm will provide an estimate about a pattern in the input data based on some input data, which can be labelled or unlabeled. An error function measures the accuracy of the model's prediction. If there are known instances, an error function can compare them to gauge the model's correctness. Weights are changed to lessen the difference between the known example and the model estimate if the model can better match the data points in the training set. This "evaluate and optimize" procedure will be repeated by the algorithm, with weights being updated automatically, until a predetermined level of accuracy is reached. Three major categories may be used to classify machine learning models.

Although the algorithm isn't trained on sample data, reinforcement machine learning is a machine learning approach that is comparable to supervised learning. By making mistakes along the way, this model learns. The optimal suggestion or strategy will be created for a specific problem by reinforcing a string of good outcomes. The algorithm learned when to try an answer (or question), which square to choose on the board, and how much to wager—especially on daily doubles—using reinforcement learning. Many machine learning algorithms are often employed.

Neural networks, which include a vast number of connected processing nodes, mimic how the human brain functions. Natural language translation, picture identification, speech recognition, and image generation are just a few of the applications that benefit from neural networks' aptitude for pattern detection. Based on a linear connection between various values, this technique is used to forecast numerical values. The method might be applied, for instance, to forecast housing values based on local historical data.

The "yes/no" responses to questions are categorical response variables, and our supervised learning method predicts them. Applications for it include sorting spam and

performing quality control on a production line. Clustering algorithms may find patterns in data to categories it via unsupervised learning. Data scientists can benefit from computers' ability to spot distinctions between data points that humans have missed.

2.3.1 Artificial Intelligence Applications

There are numerous, real-world applications of AI systems today. Below are some of the most common use cases:

- **Speech recognition:** It is also known as automatic speech recognition (ASR), computer speech recognition, or speech-to-text, and it is a capability which uses natural language processing (NLP) to process human speech into a written format. Many mobile devices incorporate speech recognition into their systems to conduct voice search—e.g. Siri—or provide more accessibility around texting.
- **Customer service:** Online virtual agents are replacing human agents along the customer journey. They answer frequently asked questions (FAQs) around topics, like shipping, or provide personalized advice, cross-selling products or suggesting sizes for users, changing the way we think about customer engagement across websites and social media platforms. Examples include messaging bots on e-commerce sites with virtual agents, messaging apps, such as Slack and Facebook Messenger, and tasks usually done by virtual assistants and voice assistants.
- **Computer vision:** This AI technology enables computers and systems to derive meaningful information from digital images, videos and other visual inputs, and based on those inputs, it can take action. This ability to provide recommendations distinguishes it from image recognition tasks. Powered by convolutional neural networks, computer vision has applications within photo tagging in social media, radiology imaging in healthcare, and self-driving cars within the automotive industry.
- **Recommendation engines:** Using past consumption behavior data, AI algorithms can help to discover data trends that can be used to develop more effective cross-selling strategies. This is used to make relevant add-on recommendations to customers during the checkout process for online retailers.
- **Automated stock trading:** Designed to optimize stock portfolios, AI-driven high-frequency trading platforms make thousands or even millions of trades per day without human intervention.
-

2.3.2 How Machine Learning Works

- **A Decision Process:** In general, machine learning algorithms are used to make a prediction or classification. Based on some input data, which can be labeled or unlabeled, your algorithm will produce an estimate about a pattern in the data.
- **An Error Function:** An error function evaluates the prediction of the model. If there are known examples, an error function can make a comparison to assess the accuracy of the model.

- **A Model Optimization Process:** If the model can fit better to the data points in the training set, then weights are adjusted to reduce the discrepancy between the known example and the model estimate. The algorithm will repeat this “evaluate and optimize” process, updating weights autonomously until a threshold of accuracy has been met.

2.3.3 Machine Learning Models fall into three primary categories

2.3.3.1 Supervised machine learning

Supervised learning, also known as supervised machine learning, is defined by its use of labeled datasets to train algorithms to classify data or predict outcomes accurately. As input data is fed into the model, the model adjusts its weights until it has been fitted appropriately. This occurs as part of the cross validation process to ensure that the model avoids overfitting or under fitting. Supervised learning helps organizations solve a variety of real-world problems at scale, such as classifying spam in a separate folder from your inbox. Some methods used in supervised learning include neural networks, naïve Bayes, linear regression, logistic regression, random forest, and support vector machine (SVM).

2.3.3.2 Unsupervised machine learning

Unsupervised learning, also known as unsupervised machine learning, uses machine learning algorithms to analyze and cluster unlabeled datasets. These algorithms discover hidden patterns or data groupings without the need for human intervention. This method’s ability to discover similarities and differences in information make it ideal for exploratory data analysis, cross- selling strategies, customer segmentation, and image and pattern recognition. It’s also used to reduce the number of features in a model through the process of dimensionality reduction. Principal component analysis (PCA) and singular value decomposition (SVD) are two common approaches for this. Other algorithms used in unsupervised learning include neural networks, k-means clustering, and probabilistic clustering methods.

2.3.3.3 Semi-supervised learning

Semi-supervised learning offers a happy medium between supervised and unsupervised learning. During training, it uses a smaller labeled data set to guide classification and feature extraction from a larger, unlabeled data set. Semi-supervised learning can solve the problem of not having enough labeled data for a supervised learning algorithm. It also helps if it’s too costly to label enough data.

2.3.4 Common Machine Learning Algorithms

A number of machine learning algorithms are commonly used. These include:

- **Neural networks:** Neural networks simulate the way the human brain works, with a huge number of linked processing nodes. Neural networks are good at recognizing patterns and play an important role in applications including natural

language translation, image recognition, speech recognition, and image creation.

Linear regression: This algorithm is used to predict numerical values, based on a linear relationship between different values. For example, the technique could be used to predict house prices based on historical data for the area.

Logistic regression: This supervised learning algorithm makes predictions for categorical response variables, such as “yes/no” answers to questions. It can be used for applications such as classifying spam and quality control on a production line.

Clustering: Using unsupervised learning, clustering algorithms can identify patterns in data so that it can be grouped. Computers can help data scientists by identifying differences between data items that humans have overlooked.

Decision trees: Decision trees can be used for both predicting numerical values (regression) and classifying data into categories. Decision trees use a branching sequence of linked decisions that can be represented with a tree diagram. One of the advantages of decision trees is that they are easy to validate and audit, unlike the black box of the neural network.

Random forests: In a random forest, the machine learning algorithm predicts a value or category by combining the results from a number of decision trees.

2.4 What is Deep Learning?

Neural network-based deep learning was first proposed decades ago. It was initially evolved by specialist Lecunetal in 1998. LeNet5, a five-layer classifier made with a Convolutional Neural Network (CNN), was his creation. Over the course of the past few years, deep learning has been able to achieve a great deal of success as a result of significant advancements in computing power and the explosion of big data. Profound learning depends on enormous information gathered in a specific field. Resources for learning from massive amounts of data are crucial. Profound implies that a brain network has loads of layers for mirroring our mind. Big data sets can now be collected, managed, and analyzed thanks to powerful computing facilities, high-performance GPUs, ASIC accelerators, cloud storage, and other innovations. Since just with informational collections sufficiently huge, can overfitting issues be settled in profound learning. Additionally, the increased computing power may be able to speed up the lengthy training process.

In computer vision and object detection, deep leaning-based approaches have a significant advantage over conventional algorithms and are increasingly being used in a variety of fields. The incorporation of deep learning has resulted in improvements to the performance of numerous robotics systems. Take Alpha Go by Google as an illustration. It studied how people learn and competed with the well-known Go player.

It is essential to have sufficient examples from previously collected images in order to foster deep learning in computer vision. One good illustration is ImageNet. One commitment of this exploration remembers fostering a data set of plant for sea water to help preparing and testing. However, a learning algorithm is also essential. Conventional PC vision and picture handling approaches experienced the exactness of component extraction, while profound learning strategy can be used to work on the procedure through brain organization.

A form of machine learning that uses multiple processing layers to extract progressively higher-level features from data. It is based on CONTs.

2.4.1 Rise of Deep Learning

Machine Learning is said to have occurred in the 1950s when Alan Turing, a British mathematician, proposed his artificially intelligent “learning machine.” Arthur Samuel wrote the first computer learning program. His program made an IBM computer improve at the game of checkers the longer it played. In the decades that followed, various machine learning techniques came in and out of fashion.

Neural networks were mostly ignored by machine learning researchers, as they were plagued by the ‘local minima’ problem in which weightings incorrectly appeared to give the fewest errors. However, some machine learning techniques like computer vision and facial recognition moved forward. In 2001, a machine learning algorithm called Ad boost was developed to detect faces within an image in real-time. It filtered images through decision sets such as “does the image have a bright spot between dark patches, possibly denoting the bridge of a nose?” When the data moved further down the decision tree, the probability of selecting the right face from an image grew.

Neural networks did not return to favor for several more years when powerful graphics processing units finally entered the market. The new hardware-enabled researchers to use desktop computers instead of supercomputers to run, manipulate, and process images. The most significant leap forward for neural networks happened because of the introduction of substantial amounts of labeled data with ImageNet, a database of millions of labeled images from the Internet. The cumbersome task of manually labeling images was replaced by crowdsourcing, giving networks a virtually unlimited source of training materials. In the years since technology companies have made their deep learning libraries open source. Examples include Google Tensorflow, Facebook open-source modules for Torch, Amazon DSSTNE on GitHub, and Microsoft CNTK.

2.4.2 Deep Learning in Action

Aside from your favorite music streaming service suggesting tunes you might enjoy, how is deep learning impacting people's lives? As it turns out, deep learning is finding its way into applications of all sizes. Anyone using Facebook cannot help but notice that the social platform commonly identifies and tags your friends when you upload new photos. Digital assistants like Siri, Cortana, Alexa, and Google Now use deep learning for natural language processing and speech recognition. Skype translates spoken conversations in real-time. Many email platforms have become adept at identifying spam messages before they even reach the inbox. PayPal has implemented deep learning to prevent fraudulent payments. Apps like Cam Find allow users to take a picture of any object and, using mobile visual search technology, discover what the object is.

Google, in particular, is leveraging deep learning to deliver solutions. Google Deep Mind’s Alpha Go computer program recently defeated standing champions at the game of Go. DeepMind’s Wave Net can generate speech mimicking human voice that sounds more natural than speech systems presently on the market. Google Translate is using

deep learning and image recognition to translate voice and written languages. Google Planet can identify where any photo was taken. Google developed the deep learning software database, Tensorflow, to help produce AI applications.

Deep learning is only in its infancy and, in the decades to come, will transform society. Self-driving cars are being tested worldwide; the complex layer of neural networks is being trained to determine objects to avoid, recognize traffic lights, and know when to adjust speed. Neural networks are becoming adept at forecasting everything from stock prices to the weather. Consider the value of digital assistants who can recommend when to sell shares or when to evacuate ahead of a hurricane. Deep learning applications will even save lives as they develop the ability to design evidence-based treatment plans for medical patients and help detect cancers early.

Now, as you have clearly understood what is Deep Learning, and want to step up in this cutting-edge technology, you must know the career prospects.

2.4.3 Example of Deep Learning at Work

Let's say the goal is to have a neural network recognize photos that contain a dog. All dogs don't look exactly alike – consider a Rottweiler and a Poodle, for instance. Furthermore, photos show dogs at different angles and with varying amounts of light and shadow. So, a training set of images must be compiled, including many examples of dog faces which any person would label as “dog,” and pictures of objects that aren't dogs, labeled (as one might expect), “not dog.” The images, fed into the neural network, are converted into data. These data move through the network, and various nodes assign weights to different elements. The final output layer compiles the seemingly disconnected information – furry, has a snout, has four legs, etc. – and delivers the output: dog.

Now, this answer received from the neural network will be compared to the human-generated label. If there is a match, then the output is confirmed. If not, the neural network notes the error and adjusts the weightings. The neural network tries to improve its dog-recognition skills by repeatedly adjusting its weights over and over again. This training technique is called supervised learning, which occurs even when the neural networks are not explicitly told what “makes” a dog. They must recognize patterns in data over time and learn on their own.

After learning what is Deep Learning, and understanding the principles of its working, let's go a little back and see the rise of Deep Learning.

2.4.4 How Deep Learning Works

- Most deep learning methods use neural network architectures, which is why deep learning models are often referred to as deep neural networks.
- The term “deep” usually refers to the number of hidden layers in the neural network. Traditional neural network only contains 2-3 hidden layers, while deep networks can have as many as 150.

- Deep learning models are trained by using large sets of labeled data and neural network architectures that learn features directly from the data without the need for manual feature extraction.
- One of the most popular types of deep neural networks is known as convolutional neural networks (CNN or Convnet). A CNN convolves learned features with input data, and uses 2D convolutional layers, making this architecture well suited to processing 2D data, such as images.
- CNNs eliminate the need for manual feature extraction, so you do not need to identify features used to classify images. The CNN works by extracting features directly from images. The relevant features are not pertained; they are learned while the network trains on a collection of images. This automated feature extraction makes deep learning models highly accurate for computer vision tasks such as object classification.
- CNNs learn to detect different features of an image using tens or hundreds of hidden layers. Every hidden layer increases the complexity of the learned image features. For example, the first hidden layer could learn how to detect edges, and the last learns how to detect more complex shapes specifically catered to the shape of the object we are trying to recognize.

2.5 Difference Between Machine Learning and Deep Learning

Deep learning is a specialized form of machine learning. A machine learning workflow starts with relevant features being manually extracted from images. The features are then used to create a model that categorizes the objects in the image. With a deep learning workflow, relevant features are automatically extracted from images. In addition, deep learning performs “end-to-end learning” – where a network is given raw data and a task to perform, such as classification, and it learns how to do this automatically.

Another key difference is deep learning algorithms scale with data, whereas shallow learning converges. Shallow learning refers to machine learning methods that plateau at a certain level of performance when you add more examples and training data to the network.

A key advantage of deep learning networks is that they often continue to improve as the size of your data increases.

2.5.1 Two main key points are used in both type of learning

- Feature Extraction
- Feature Selection

It means we extract all the features from given problem and then we select most important features from the given problem.

5 Key differences between machine learning and deep learning

While there are many differences between these two subsets of artificial intelligence, here are five of the most important.

1. Human Intervention

Machine learning requires more ongoing human intervention to get results. Deep learning is more complex to set up but requires minimal intervention thereafter.

2. Hardware

Machine learning programs tend to be less complex than deep learning algorithms and can often run on conventional computers, but deep learning systems require far more powerful hardware and resources. This demand for power has driven has meant increased use of graphical processing units. GPUs are useful for their high bandwidth memory and ability to hide latency (delays) in memory transfer due to thread parallelism (the ability of many operations to run efficiently at the same time.)

3. Time

Machine learning systems can be set up and operate quickly but may be limited in the power of their results. Deep learning systems take more time to set up but can generate results instantaneously (although the quality is likely to improve over time as more data becomes available).

4. Approach

Machine learning tends to require structured data and uses traditional algorithms like linear regression. Deep learning employs neural networks and is built to accommodate large volumes of unstructured data.

5. Applications

Machine learning is already in use in your email inbox, bank, and doctor's office. Deep learning technology enables more complex and autonomous programs, like self-driving cars or robots that perform advanced surgery.

2.6 Transfer Learning

Transfer learning is the process of using a machine learning model that has already been trained to solve a separate but connected problem. For instance, if you trained a straightforward classifier to identify the presence of a backpack in a picture, you might utilize the information the model learned during training to identify additional items like sunglasses. The core idea behind transfer learning is to use what has been learnt in one activity to enhance generalization in another. We move the weights that a network has picked up at "task A" to a fresh "task B". The main concept is to apply what a model has learnt from a task with a lot of labelled training data to a new task with little to no training data. We begin the learning process using patterns discovered while completing a comparable activity, as opposed to starting from scratch. Due to the enormous amount of CPU power needed, transfer learning is mostly employed in computer vision and natural language processing applications like sentiment analysis. Transfer learning is more of a "design methodology" within the subject of active learning than it is a machine learning technology. Additionally, it is not a specialized branch of research within machine learning. However, it has gained a lot of traction when used with neural networks, which demand a lot of data and processing power.

Neural networks, for instance, frequently attempt to identify edges in the earlier layers, forms in the middle layer, and certain task-specific properties in the latter layers in computer vision. The early and intermediate layers are utilized in transfer learning, whereas the latter layers are just retrained. It aids in using the labelled data from the first job it was trained on. Let's return to the example of the model that will be used to recognize sunglasses.

Trained to recognize a bag on a picture. We will merely retrain the later layers so that they may learn what distinguishes sunglasses from other things because the model has already learnt to recognize objects in the earlier levels. Transfer learning is the process of transferring as much information as feasible from the task the model was trained on to the current task. Depending on the issue and the information, this knowledge might take many different forms. For instance, it may be the way models are put together, which makes it simpler for humans to recognize fresh items. Although transfer learning has other advantages as well, its key advantages include reducing training time, improving neural network performance (in most circumstances), and using less data.

Transfer learning is useful in situations when access to large amounts of data is not always possible yet is required to train a neural network from start. Because the model has previously been trained, transfer learning allows for the construction of reliable machine learning models with relatively minimal training data. This is particularly useful in natural language processing since producing sizable labelled data sets typically necessitates expert expertise. Because it can occasionally take days or even weeks to train a deep neural network from scratch on a tough problem, training time is also shortened.

With the aid of artificial intelligence, computer vision programs have accomplished remarkable feats that have proven to be of significant use to society. Machine learning algorithms and data mining have achieved miraculous success in a number of knowledge engineering disciplines, including regression, clustering, classification, etc. The vast majority of machine learning and AI models assume that training and test data come from the same feature space or distribution. It will be necessary to develop a whole new statistical model from the ground up by training it with freshly acquired data if any modifications are made to the distribution space. In the majority of real-world applications, this approach is time-consuming and most definitely costly. It is also challenging, and it might be quite difficult in some circumstances to gather the necessary fresh, pertinent training data for recreating the model. If there is less of a requirement to gather training data at these times, it would be a huge relief. Transfer learning will assist in achieving this. It can also be summed up as the transfer of knowledge. Transfer learning is the process of using the information acquired from one work to complete other activities that are connected to it.

The solitary learning paradigm of the standard deep learning and machine learning algorithms is then overcome thanks to this. Transfer learning is really a "design methodology" as opposed to a machine learning method. One of the most promising fields, it may also the creation of artificial general intelligence (AGI), according to Demis Hassabis, CEO of DeepMind. There are several instances where the idea of transfer learning may be quite helpful. One such instance would be Wi-Fi-based interior localization, where the information might quickly become out of date. The objective in this case is to determine a person's current position using Wi-Fi data that has already been gathered in the past. Because each site must identify a massive quantity of signal

data, calibrating Wi-Fi data to create transfer learning models in a large-scale setting is quite costly. The performance of location estimation on different devices or time periods may be decreased by a model that was trained for that device or time period. I'm here to make it easier. The pre trained model trained in the source domain may benefit from a model recalibration to adapt to the destination domain. Fig. 2.3 describes how transfer learning mechanism works it overrides the pre trained model weights and learns using its weights.

Trained to spot a bag on a photograph. The model has already learned to recognise objects in the earlier levels; we will simply retrain the latter layers so that they may learn what sets sunglasses apart from other objects. The process of transferring as much knowledge as is practical from the task the model was trained on to the current task is known as transfer learning. This knowledge may be expressed in a variety of ways, depending on the situation and the available information. For instance, it might be the construction of models that facilitates human recognition of new objects. Although transfer learning also offers additional benefits, its main benefits are shorter training times, better neural network performance (in most cases), and less data usage.

The concept of transfer learning can be quite beneficial in a variety of situations. An example of this would be Wi-Fi-based interior localisation, where the data could easily become outdated. Using Wi-Fi data that has previously been obtained in the past, the goal in this scenario is to determine a person's current location. It is quite expensive to calibrate Wi-Fi data to develop transfer learning models in a large-scale environment since each site must identify a huge amount of signal data. A model that was trained for a certain device or period of time may perform location estimate less accurately on other devices or at various times. For your convenience, I'm here. The source trained the pre-trained model.

2.7 Neural Network

Throughout recent years, brain networks in profound learning were getting progressively well known. In the ImageNet Large Scale Visual Recognition Challenge, CNN was used to classify images, and the accuracy of the test was significantly higher than that of traditional algorithms. This achievement has raised interest in neural network-based deep learning. In 2014, Ross et al. proposed a calculation called Quick R-CNN which intends to change over object ID into a relapse issue. When compared to the previous best result of 53.3% in the ImageNet Large Scale Visual Recognition Challenge in 2012, the mean average precision increased by nearly 30%.

Because features from a variety of sizes of thousands of proposals in each image would be extracted, the amount of computation required was enormous. In computer vision, which involves target detection, image classification, and object identification, Faster R-CNN has recently gained widespread adoption due to its significant reduction in computational requirements. Facebook's YOLO proposal also marks a turning point in corresponding research.

Each neuron-to-neuron connection has a corresponding weight, which establishes its potency and effect. The network's performance on a particular task, like classification or regression, is optimized during training by iteratively adjusting these weights. A

subfield of machine learning called deep learning makes use of artificial neural networks (ANNs) with many hidden layers. Because these deep neural networks have the ability to learn hierarchical data representations, they can recognize intricate patterns and relationships. Deep learning has achieved state-of-the-art performance in tasks including image classification, object identification, and picture segmentation, revolutionizing a variety of fields, particularly computer vision. The capacity of convolutional neural networks (CNNs) to efficiently analyze and extract information from pictures makes them a popular choice for computer vision jobs.

2.7.1 Types of neural networks

Neural networks can be classified into different types, which are used for different purposes. While this isn't a comprehensive list of types, the below would be representative of the most common types of neural networks that you'll come across for its common use cases:

The perceptron is the oldest neural network, created by Frank Rosenblatt in 1958. Feedforward neural networks, or multi-layer perceptrons (MLPs), are what we've primarily been focusing on within this article. They are comprised of an input layer, a hidden layer or layers, and an output layer. While these neural networks are also commonly referred to as MLPs, it's important to note that they are actually comprised of sigmoid neurons, not perceptrons, as most real-world problems are nonlinear. Data usually is fed into these models to train them, and they are the foundation for computer vision, natural language processing, and other neural networks.

Convolutional neural networks (CNNs) are similar to feedforward networks, but they're usually utilized for image recognition, pattern recognition, and/or computer vision. These networks harness principles from linear algebra, particularly matrix multiplication, to identify patterns within an image.

Recurrent neural networks (RNNs) are identified by their feedback loops. These learning algorithms are primarily leveraged when using time-series data to make predictions about future outcomes, such as stock market predictions or sales forecasting.

2.8 Conventional Neural Network

Fake Brain Organizations, truncated as ANNs can be depicted as computational frameworks that are intended to imitate the human mind's examination and data handling abilities, and very much like the human cerebrum an ANN comprises of a coordinated chart with interconnected handling components known as neurons (Jain, Mao and Mohiuddin,1996). There are various kinds of brain organizations like the Repetitive Brain Organization (RNN), Multi-facet Perceptron (MLP), Convolutional Brain Organization (CNN), and so forth. Among the numerous Brain Organizations present the most well-known one is the Multi-Facet Perceptron organization (MLP).

A run of the mill MLP network includes different hub layers, specifically, an information layer, at least one secret layers lastly a result layer. The hubs are all

associated with one another and every hub is related with a weight and limit. The weight alludes to the significance that two hubs interfacing have in an organization and information is moved starting with one layer then onto the next provided that the result of a singular hub is over the edge determined (Jain, Mao and Mohiuddin,1996).

Convolutional Neural Networks Regular neural networks like the Multi-layer Perceptron (MLP) were once used for image classification. However, as the resolution of the images being classified increased, the networks became computationally difficult to manage and the total number of parameters used for classification would become excessive. Convolutional Brain Organizations are basically the same in working to customary brain organizations like the MLP, in any case, what changes in Convolutional Brain Organizations is that the layers of a CNN have three-layered game plan (width, level and profundity) of neurons rather than the standard two-layered cluster and for this straightforward explanation CNNs are broadly involved on picture information with the end goal of grouping as the engineering of a CNN is intended to exploit the 3d type of a picture. A basic Convolutional Brain Organization's engineering comprises of three fundamental layers, in particular, Convolutional layer, pooling layer and the Completely associated layer.

A CNN's main building block is the Convolutional layer, which is made up of learnable parameters called filters or kernels. The filter is in charge of finding patterns in the input image, such as textures, edges, shapes, and objects, among other things. Each channel slides/convolves over the level and width of the information picture, processing the dab item between the channel and the pixels present in the information picture. A feature map that summarizes all of the features found in the input image is the Convolutional layer's resultant (Yamashita et al., 2018). Another component of a CNN is the Pooling layer, which is used for down sampling to reduce the feature map's spatial size.

Neural networks with convolutions For image categorization, conventional neural networks like the Multi-layer Perceptron (MLP) were originally employed. Although the networks grew computationally challenging to manage and the overall number of parameters employed for classification would become excessive as the resolution of the images being classed rose. However, what distinguishes Convolutional Brain Organisations from conventional brain organisations like the MLP is that the layers of a CNN have a three-layered game plan (width, level, and profundity) of neurons instead of the conventional two-layered cluster. For this simple explanation, CNNs are heavily focused on picture information with the goal of grouping as the engineering of a CNN is intricate.

This is done to control overfitting and reduce the number of parameters and computations in the network. In CNN, there are two distinct pooling methods: max pooling and average pooling. Average Pooling returns the average of all the values present in the kernel-convolved portion of the image, while Max Pooling returns the maximum value in that region (Yamashita et al., 2018). The Rectified Linear Unit (ReLU) activation function used to introduce non-linearity into the network is typically followed by this layer, which transforms all negative values in the feature map into zeros.

Because it contributes to increasing the images' non-linearity, the inclusion of ReLU in a CNN is crucial. However, when imposing an image through a Convolutional layer to create a feature map, there may be some linearity introduced in the image. In order to bring back the non-linearity in the image, the ReLU activation function is used (Zhao et al., 2018). Images as such are known to contain non-linear features such as different objects or the boundaries between those objects. The architecture receives its first introduction of fully connected layers following the addition of a number of Pooling and Convolutional layers.

The information coming to the completely associated layer is the straightened yield from the last convolutional and pooling layer. In order for the fully connected layers, which carry out the same operations as an ANN to carry out the final classification and compute the class scores, the flattening is performed to convert the 3d matrix data from the final pooling layer into a 1d array of vectors (Yamashita et al., 2018). A straightforward Convolutional Neural network can read and classify images in this manner.

Forward propagation, in which the input data is sent down the layers, and backpropagation, in which the network modifies its weights in response to the calculated error, are both involved in training a CNN. The network may learn from this process and refine its settings to reduce the discrepancy between expected and actual results. CNNs have completely changed the field of computer vision and excel at a wide range of tasks. They are highly suited for jobs that involve comprehending and analyzing visual input due to their natural learning of hierarchical data representations and their translation-invariant characteristic. Convolution is the process of combining two functions to produce the output of the other function. The input image is convoluted with the application of filters in CNNs, resulting in a Feature map. Filters are weights and biases that are randomly generated vectors in the network. Instead of having individual weights and biases for each neuron, CNN uses the same weights and biases for all neurons. Many filters can be created, each of which catches a different aspect from the input. Kernels are another name for filters. Fig. 2.6 describes the working of convolutional neural network. It takes the input images and the input layers extract the feature map from input image and then it is passed to the hidden layers. The hidden layers contain different convolution, max pooling and pooling layers. The features of input images are processed in the hidden layer for the classification of input image. After processing the hidden layers passed the results to the output layer and it classifies the input image showing results.

The straightened yield from the final convolutional and pooling layer is the data entering the fully associated layer. Flattening is done to change the 3d matrix data from the last pooling layer into a 1d array of vectors so that the fully connected layers, which conduct the same operations as an ANN, may carry out the final classification and compute the class scores (Yamashita et al., 2018). This may be done using a simple Convolutional Neural network to interpret and categorise photos.

Training a CNN involves both forward propagation, in which the input data is passed through the layers, and backpropagation, in which the network adjusts its weights in response to the computed error. This procedure might teach the network something.

2.8.1 Applications

- Below are some applications of Convolutional Neural Networks used today:
- Object detection: With CNN, we now have sophisticated models like R-CNN, Fast R-CNN, and Faster R-CNN that are the predominant pipeline for many object detection models deployed in autonomous vehicles, facial detection, and more.
- Semantic segmentation: In 2015, a group of researchers from Hong Kong developed a CNN-based Deep Parsing Network to incorporate rich information into an image segmentation model. Researchers from UC Berkeley also built fully convolutional networks that improved upon state-of-the-art semantic segmentation.
- Image captioning: CNNs are used with recurrent neural networks to write captions for images and videos. This can be used for many applications such as activity recognition or describing videos and images for the visually impaired. It has been heavily deployed by YouTube to make sense to the huge number of videos uploaded to the platform on a regular basis.
-

2.8.2 Motivation behind Convolution

Convolution leverages three important ideas that motivated computer vision researchers: sparse interaction, parameter sharing, and equivariant representation. Let's describe each one of them in detail. Trivial neural network layers use matrix multiplication by a matrix of parameters describing the interaction between the input and output unit. This means that every output unit interacts with every input unit. However, convolution neural networks have *sparse interaction*. This is achieved by making kernel smaller than the input e.g., an image can have millions or thousands of pixels, but while processing it using kernel we can detect meaningful information that is of tens or hundreds of pixels. This means that we need to store fewer parameters that not only reduces the memory requirement of the model but also improves the statistical efficiency of the model. If computing one feature at a spatial point (x_1, y_1) is useful then it should also be useful at some other spatial point say (x_2, y_2) . It means that for a single two-dimensional slice i.e., for creating one activation map, neurons are constrained to use the same set of weights. In a traditional neural network, each element of the weight matrix is used once and then never revisited, while convolution network has *shared parameters* i.e., for getting output, weights applied to one input are the same as the weight applied elsewhere. Due to parameter sharing, the layers of convolution neural network will have a property of equivariance to translation. It says that if we changed the input in a way,

the output will also get changed in the same way. A convolution network contains shared parameters, meaning that weights applied to one input are the same as weights applied elsewhere for getting output, unlike a standard neural network where each member of the weight matrix is utilised just once and never again. The layers of a convolution neural network will have the attribute of equivariance to translation as a result of parameter sharing. It asserts that if we adjust the input in a certain way, the output will follow suit.

. Pooling Layer

The pooling layer replaces the output of the network at certain locations by deriving a summary statistic of the nearby outputs. This helps in reducing the spatial size of the representation, which decreases the required amount of computation and weights. The pooling operation is processed on every slice of the representation individually. There are several pooling functions such as the average of the rectangular neighborhood, L2 norm of the rectangular neighborhood, and a weighted average based on the distance from the central pixel. However, the most popular process is max pooling, which reports the maximum output from the neighborhood.

By calculating an aggregate statistic from the surrounding outputs, the pooling layer substitutes for the network's output at specific locations. This aids in shrinking the representation's spatial size, which lowers the amount of computation and weights needed. Each slice of the representation is subjected to the pooling operation separately. A weighted average based on the distance from the central pixel is one of the pooling functions, along with the average of the rectangular neighborhood and the L2 norm of the rectangle neighborhood. However, max pooling, which reports the highest output from the neighborhood, is the most widely used technique.

- **Fully Connected Layer**

Neurons in this layer have full connectivity with all neurons in the preceding and succeeding layer as seen in regular FCNN. This is why it can be computed as usual by a matrix multiplication followed by a bias effect. The FC layer helps to map the representation between the input and the output. As in a conventional FCNN, all of the neurons in this layer are fully connected to all of the neurons in the layer before and after. Because of this, it can be calculated using a matrix multiplication followed by a bias effect, as per usual. The representation between the input and the output is mapped using the FC layer.

- **Non-Linearity Layers**

Since convolution is a linear operation and images are far from linear, non-linearity layers are often placed directly after the convolutional layer to introduce non-linearity to the activation map.

2.9 Available Methodology

Different criteria are used to classify research methods. They are a general class, nature of review and reason for review and exploration plan. Additionally, case studies and interviews are based on research methodology. Numerous techniques have been developed to find plant and keep track of their number and size. Different object detection techniques have been used to detect plant. Image processing and computer vision systems are taken into consideration to track plant size and count. Tracking technologies like optical flow and frame subtraction are used to monitor the movement of plant.

A goal was to develop a model that uses object detection algorithms to automatically characterize the movie. For a human to explain a video's content is a simple assignment, but it is a hard and challenging task.

This study introduces a CNN model with picture segmentation to find plant in ocean water that is foggy. A particular data set was produced to help in this investigation. The data augmentation alteration approach was used to obtain more learning materials because the original images in the particular setting are insufficient for training objectives. To deal with the overfitting issue, it employs the dropout method. Because our goal is to integrate this system into an AUV, which needs real-time applications, some trade-offs were taken into account in order to speed up processing. The specific system design and optimization methods are covered in this section.

Research methodologies are categorised according to many criteria. They belong to a general class, with a general review and exploration strategy. Case studies and interviews are additionally based on research technique. Many methods have been developed to locate plant and monitor their quantity and size. plant detection has been done using a variety of object detection methods. To keep track of plant size and count, image processing and computer vision systems are used. plant movement is tracked using tracking technologies like optical flow and frame subtraction.

In order to detect plant in ocean water that is hazy, this research introduces a CNN model with image segmentation. To assist with this investigation, a specific data set was created. The original photos in the specific environment are insufficient for training purposes, so the data augmentation alteration strategy was employed to gain more learning materials. It uses the dropout technique to address the overfitting issue. Some trade-offs were considered to speed up processing because our aim is to integrate this system into an AUV, which necessitates real-time applications. This section discusses the detail system design and optimization techniques.

2.10 Research Work

Agriculture remains the backbone of many economies around the world, especially in developing countries where a significant portion of the population relies on it for livelihood. One of the major challenges farmers face is the early and accurate identification of plant diseases. Delayed or incorrect diagnosis often leads to poor crop yield, economic losses, and the overuse of pesticides. Traditional disease detection methods rely heavily on manual inspection by agricultural experts, which is not always feasible, especially in rural areas where access to specialists is limited. To address this problem, this research focuses on developing an automated plant disease detection system using deep learning techniques, specifically Convolutional Neural Networks (CNNs), which have proven highly effective in image classification tasks. The workflow for this system follows a logical sequence: training the plant disease dataset, building the model, compiling it, evaluating its performance, saving and visualizing the model, analyzing evaluation metrics, making predictions on single images, and testing the model's overall functionality.

The process starts with the **training phase**, where a large collection of labeled images of plant leaves is used. This dataset typically includes images of healthy leaves and those affected by various diseases.

Once the data is ready, the next step involves **building the model** using a deep learning architecture. A Convolutional Neural Network (CNN) is used, which consists of several layers such as convolutional layers for feature extraction, pooling layers for down-sampling, and dense layers for classification.

After defining the architecture, the model is **compiled**. This step involves setting the optimizer, loss function, and metrics, which is ideal for multi-class classification problems. Accuracy is set as the primary metric to evaluate how well the model predicts disease labels. Following compilation, the model undergoes **evaluation** using the validation dataset.

To preserve the trained model for future use, it is **saved** in a file format such as .h5. Saving the model avoids the need to retrain it every time, which is particularly useful for deployment in real-world applications like mobile or web apps used by farmers.

An important aspect of this project is **visualizing accuracy and loss** over training epochs. Line graphs are generated to display the progression of training and validation accuracy. If both curves steadily increase and stabilize, it indicates that the model has learned meaningful patterns. Similarly, a plot of training and validation loss helps identify any potential overfitting.

Beyond accuracy and loss, **other evaluation metrics** are calculated to assess model performance in greater detail. These include precision, recall, and F1-score. Precision measures the accuracy of positive predictions, while recall indicates how well the model detects all positive cases. The F1-score is a harmonic mean of precision and recall and is especially useful when dealing with imbalanced datasets.

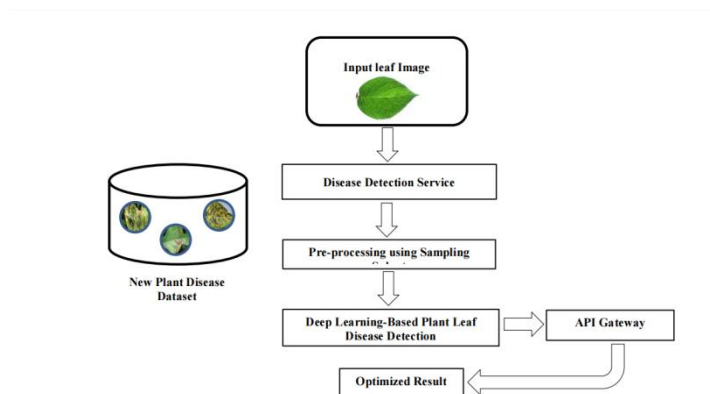
Another essential evaluation tool is the **confusion matrix**, which displays the number of correct and incorrect predictions for each disease class. A well-performing model will have a strong diagonal in the matrix, representing correct predictions, and very few off-diagonal elements.

After training and evaluation, the model is **loaded** from the saved file. This step restores the model's learned parameters and structure,.

A key practical feature of this project is the ability to **perform predictions on single images**. A new image of a plant leaf is loaded, preprocessed to match the input format of the model, and passed through the network to predict the most likely disease. The prediction is visualized by displaying the input image along with the predicted disease name and the model's confidence score.

Finally, the model is **tested** on a separate test dataset that it has never seen before. This step confirms whether the model performs consistently across new data. The test accuracy and other metrics provide a final benchmark for how effective the model is in real-world scenarios.

The plant disease detection system developed using deep learning provides an efficient, accurate, and scalable solution for identifying crop diseases from leaf images. By following a structured workflow that includes training, model building, evaluation, visualization, and prediction, the project successfully demonstrates the practical application of artificial intelligence in agriculture, with the potential to support early disease detection and improve crop management for farmers.



2.11 Chosen Methodology

Exploration research is the method chosen because it involves investigating a problem that has previously been studied or thoroughly investigated. In exploratory research, a more specific problem is formed by familiarizing oneself with an existing phenomenon and gaining new insight into it.

Because it entails looking into an issue that has already been explored or completely investigated, exploration research was chosen as the approach. In exploratory research, a more focused issue is created by becoming familiar with an existing occurrence and obtaining fresh understanding of it.

CHAPTER NO 03

DATA SET PREPARTION

3.1 Data Collection

Data collection is one of the most important steps in developing an effective deep learning model, especially in image-based classification tasks like plant disease detection. For this project, we collected a large and diverse dataset from multiple online sources and repositories, including publicly available datasets on platforms like GitHub and Kaggle. These datasets include real images of plant leaves affected by different types of diseases as well as healthy leaves. The collected data helps the model learn to differentiate between healthy and diseased conditions across a wide variety of crops.

In total, our dataset consists of **70,295 RGB images**, categorized into **38 classes**. Each class represents a unique combination of plant type and disease condition. For example, some classes represent specific diseases affecting apple plants, while others include healthy leaves of crops like corn and blueberry. This variety ensures that the model can learn detailed visual features that are specific to each plant and disease. The dataset includes images collected under different lighting conditions, backgrounds, and orientations, which helps in building a robust model.

Here are some of the class names from the dataset:

- Apple_Apple_scab
- Apple_black_rot
- Apple_healthy
- Blueberry_healthy
- Cherry_Powdery_mildew
- Cherry_healthy
- Corn_Cercospora_leaf_spot
- Corn_healthy
- Grape_Black_rot
- Grape_Esca
- Grape_leaf_blight
- Grape_healthy

This well-structured and diverse dataset forms the foundation of our deep learning model and plays a crucial role in accurately detecting and identifying plant diseases across a wide range of species.

3.2 Train Plant Disease

Training the plant disease detection system is a crucial step in building an intelligent model capable of identifying various plant diseases accurately. In this phase, we used the preprocessed dataset containing **70,295 labeled images** across **38 different plant classes**, which include both healthy and diseased leaves. The dataset was split into training, validation, and testing sets to ensure the model could learn effectively and be evaluated properly.

We used **Python** as the primary programming language and implemented the model using **TensorFlow** and **Keras**, which are popular libraries for building deep learning models. The core of our system is a **Convolutional Neural Network (CNN)**, which is well-suited

for image classification tasks. The CNN architecture allows the model to automatically extract important visual features like edges, textures, and patterns from the leaf images, enabling it to distinguish between different diseases.

During training, the model adjusted its internal parameters over multiple epochs, gradually improving its ability to classify images. We used optimization techniques like the **Adam optimizer** and **categorical cross-entropy** as the loss function, along with **accuracy** as the main performance metric. Training was conducted on batches of images, and after each epoch, we evaluated the model's performance using the validation set to monitor overfitting and learning progress.

As a result of training, the model achieved high accuracy in identifying plant diseases from unseen images. The training process also generated logs and metrics, such as accuracy and loss curves, which we later visualized to understand how well the model was learning over time. In conclusion, the training phase successfully enabled our system to recognize and classify plant diseases with a high level of confidence, providing a reliable tool for early disease detection in agriculture.

3.3 Work Plan

Table 13.1 WORK PLAN FOR PLANT DISEASE RECOGNITION

Serial no.	Activities	Days	Description
1	Recent Research	15	We have looked over all of the most recent papers that use different trained neural networks to classify the plantes.
2	Data collect	25	The kind of information we gather is to gather pictures of various plantes..
3	Training of Data set	30	To improve accuracy, we trained our datasets many times in various ways.
4	The Model	3	After extracting features from an input image, we classify it using our trained features. We Info various pictures a few times to test our model in light of preparing.

3.4 Building Model

In deep learning projects like plant disease detection, building a model is one of the most important steps. It is the stage where we define the structure of the neural network that will learn from the input images. For our project, we used the **Keras** library, which is part of TensorFlow and is widely used for building and training deep learning models due to its simplicity and flexibility. The method we followed to build the model is called **Sequential**, which allows us to add one layer at a time in a linear fashion.

Our model was trained to identify plant diseases from leaf images. To make this possible, we used different layers from the Keras library, such as **Conv2D**, **MaxPooling2D**, **Flatten**, and **Dense**. These layers work together to extract features from the input image and make a classification. Think of each of these layers like elements in a **CSS file**—each one has a specific job in designing the final output, just like how CSS styles different parts of a webpage.

The first and most important layer in our model is the **Conv2D** layer. This layer applies filters (also called kernels) to the input image to detect patterns like edges, textures, and shapes. The filters slide across the image and create feature maps. These feature maps are then used in the next layers to learn more complex features. For our images, we used a standard size of **(128x128)** pixels, which is a good balance between image detail and processing speed.

After each Conv2D layer, we used a **MaxPooling2D** layer. This layer helps reduce the size of the feature maps while keeping the most important information. It works by taking the maximum value from a small area of the feature map. This helps in reducing computational load and also helps the model focus on the strongest features. Together, Conv2D and MaxPooling2D layers form the **convolutional part** of the model, which is responsible for learning features from images.

Once we extracted enough features using convolution and pooling layers, we used a **Flatten** layer to convert the 2D feature maps into a 1D array. This is necessary before passing the data to the dense layers. The Flatten layer acts like a bridge between the feature extraction part and the decision-making part of the model.

Then we added one or more **Dense** layers. These are fully connected layers where every neuron is connected to all the neurons in the previous layer. Dense layers take the features and perform classification. In our final layer, we used a Dense layer with a **softmax** activation function to give us the probability of each class (i.e., each type of disease or a healthy leaf).

We trained this model **four to five times**, adjusting parameters such as the number of filters, layer sizes, and learning rates. This repetition helped us improve accuracy and reduce overfitting. Each time, the model learned better from the data and made more accurate predictions.

In conclusion, the model was built step-by-step using the Keras Sequential API, with each layer (Conv2D, MaxPooling2D, Flatten, Dense) serving a key function, similar to how CSS styles and arranges elements in a web page. This structure allowed the model to effectively learn from images and detect plant diseases.

rgIn deep learning projects like plant disease detection, building a model is one of the most important steps. It is the stage where we define the structure of the neural network that will learn from the input images. For our project, we used the Keras library, which is part of TensorFlow and is widely used for building and training deep learning models due to its simplicity and flexibility. The method we followed to build the model is called Sequential, which allows us to add one layer at a time in a linear fashion.

Our model was trained to identify plant diseases from leaf images. To make this possible, we used different layers from the Keras library, such as Conv2D, MaxPooling2D, Flatten, and Dense. These layers work together to extract features from the input image and make a classification. Think of each of these layers like elements in a CSS file—each one has a specific job in designing the final output, just like how CSS styles different parts of a webpage.

The first and most important layer in our model is the Conv2D layer. This layer applies filters (also called kernels) to the input image to detect patterns like edges, textures, and shapes. The filters slide across the image and create feature maps. These feature maps are then used in the next layers to learn more complex features. For our images, we used a standard size of (128x128) pixels, which is a good balance between image detail and processing speed.

After each Conv2D layer, we used a MaxPooling2D layer. This layer helps reduce the size of the feature maps while keeping the most important information. It works by taking the maximum value from a small area of the feature map. This helps in reducing computational load and also helps the model focus on the strongest features. Together, Conv2D and MaxPooling2D layers form the convolutional part of the model, which is responsible for learning features from images.

Once we extracted enough features using convolution and pooling layers, we used a Flatten layer to convert the 2D feature maps into a 1D array. This is necessary before passing the data to the dense layers.

Then we added one or more Dense layers. These are fully connected layers where every neuron is connected to all the neurons in the previous layer. Dense layers take the features and perform classification. In our final layer, we used a Dense layer with a softmax activation function to give us the probability of each class (i.e., each type of disease or a healthy leaf).

We trained this model four to five times, adjusting parameters such as the number of filters, layer sizes, and learning rates. This repetition helped us improve accuracy and reduce overfitting. Each time, the model learned better from the data and made more accurate predictions.

In conclusion, the model was built step-by-step using the Keras Sequential API, with each layer (Conv2D, MaxPooling2D, Flatten, Dense) serving a key function, similar to how CSS styles and arranges elements in a web page. This structure allowed the model to effectively learn from images and detect plant diseases.

Chapter 04

PLANNING PROJECT

4.1 Introduction

The information needed for our entire task's study and analysis is needed during the planning phase. To get project worker support, procurement exercises are performed. We divide the planning phase into subtask and specific task phases. The project's scope is defined during the planning phase. Define a realistic budget that will enable all of the planned work to be completed, as well as the total cost of the project, which will be evaluated. In addition, sufficient details about the requirements that are necessary for the project's development will be specified. The project's milestones were previously represented by the planning phase.

4.2 Research Methodology

The term "research methodology" refers to the discussion of a researcher's methodical approach to the design and conduct of a study in order to guarantee valid and trustworthy results that meet the researcher's goals. The methods we used in our research are described in research methodology. In order to evaluate the accuracy score of our trained datasets, we should be required to explain what we did and how we did it in research methods. The higher the accuracy score, the better the results. A research methodology gives research legitimacy and provides scientifically sound findings. It also provides a detailed plan that helps to keep researchers on track, making the process smooth, effective and manageable.

During the planning stage, we require the data for our full task's study and analysis. Exercises in procurement are carried out to secure the cooperation of project workers. Subtask and particular task phases make up the planning phase. The planning stage is when the project's scope is established. Establish a reasonable budget that will allow for the completion of all the planned work and the project's overall cost, which will be assessed. Additionally, appropriate information regarding the specifications required for the project's development will be provided. Previously, the planning stage served as a representation of the project's milestones.

Having a sound research methodology in place provides the following benefits:

- Other researchers who want to replicate the research have enough information to do so.
- It can help provide researchers with a specific plan to follow throughout their research.
- The methodology design process helps researchers select the correct methods for the objectives.
- It allows researchers to document what they intend to achieve with the research from the outset.

4.2.1 Model Evaluation

4.2.1.1 Qualitative

Qualitative research involves collecting and analyzing written or spoken words and textual data. It may also focus on body language or visual elements and help to create a detailed description of a researcher's observations. Researchers usually gather qualitative data through interviews, observation and focus groups using a few carefully chosen participants. This research methodology is subjective and more time-consuming than using quantitative data. Researchers often use a qualitative methodology when the aims and objectives of the research are exploratory. For example, when they perform research to understand human perceptions regarding an event, person or product.

Quantitative

Researchers usually use a quantitative methodology when the objective of the research is to confirm something. It focuses on collecting, testing and measuring numerical data, usually from a large sample of participants. They then analyze the data using statistical analysis and comparisons.

Popular methods used to gather quantitative data are:

Survey

Questionnaires

Test

Data Base

Organizational Record

4.3 Saving Model

Once the model has been trained and evaluated, the next important step is to **save the model** so it can be used later without retraining. In our project, we used the **Keras** library to save the trained model. This allows us to preserve all the learned weights, structure, and configurations. We saved the model using the filename "**TRAINED_MODEL.KERAS**". This file acts like a container that stores the entire model, making it easy to load and use it again in the future for predictions.

After training the model, we also examined the training performance through the **training history**. Keras provides this data through the `training_history.history` object. This dictionary contains useful information about how the model performed during each epoch of training.

Inside the history, we looked at several key metrics:

- **Accuracy**: This shows how accurately the model was able to classify the training images.
- **Loss**: This tells us how much error the model made during the training process.
- **Val_accuracy** (Validation Accuracy): This metric indicates how well the model performed on unseen validation images.
- **Val_loss** (Validation Loss): This shows the error on the validation dataset.

By analyzing these values, we can understand if the model is improving or overfitting. Finally, we used the `.keys()` method to print out the available metrics stored in the history object. The

output includes: **accuracy**, **val_accuracy**, **loss**, and **val_loss**. These metrics are essential for reviewing model performance and deciding if further tuning or training is needed.

4.4 Block Diagram

Creation of images from videos using video to frame converters. Added some images in dataset collection by capturing. Collect data from Market. Collect data from sources of plants.

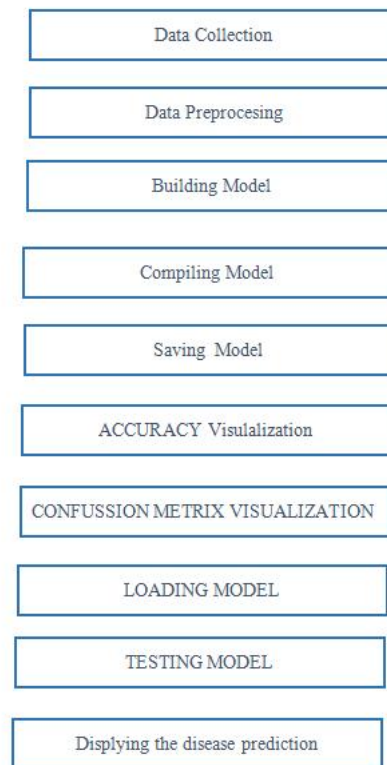


Figure 4.1 Block Diagram

Data Collection

The first and most important step in any machine learning or deep learning project is gathering a high-quality and diverse dataset. In our project, the dataset comprises **17,572 images** spanning across **38 distinct classes**, each representing a specific plant disease or a healthy plant state. These images were gathered from various online datasets such as the PlantVillage dataset and agricultural research databases.

Each image in the dataset is labeled, enabling supervised learning where the model learns the mapping between image features and disease labels. The dataset includes images of leaves affected by bacterial, fungal, and viral diseases, as well as healthy leaves. A diverse dataset like this ensures that the trained model can generalize well across different plant types, lighting conditions, leaf angles, and backgrounds.

Data Preprocessing

Raw image data is rarely suitable for direct input into a deep learning model. Therefore, we conducted several preprocessing steps to prepare the dataset for training:

- **Resizing Images:** All images were resized to a consistent dimension to match the input size required by our convolutional layers. This also helps in reducing computational cost.
- **Normalization:** Pixel values of images were scaled to a range between 0 and 1. Normalization speeds up convergence during training and stabilizes gradients.
- **Label Encoding:** The disease classes (text labels) were converted into numeric format using label encoding or one-hot encoding to be understood by the neural network.
- **Splitting the Dataset:** The dataset was divided into three subsets:
- **Training Set:** Used to train the model (typically 70-80% of data)
- **Validation Set:** Used to fine-tune model hyperparameters and monitor overfitting
- **Test Set:** Used to evaluate final model performance on unseen data

Preprocessing ensures data uniformity, enhances training quality, and provides the model with diverse examples of each class.

Building the Model

Model construction is the backbone of deep learning-based disease detection. We used **Keras** (a high-level neural network API running on top of TensorFlow) and opted for a **Sequential** model, which is ideal for stacking layers linearly.

The model architecture includes the following components:

-The layers are arranged sequentially, forming a CNN that learns hierarchical features. The model was trained in multiple rounds (4–5 iterations), tweaking layers and parameters such as the number of filters, kernel size, and activation functions.

Dropout layers were optionally added to reduce overfitting, and **ReLU (Rectified Linear Unit)** was used as the activation function in convolutional layers. The final Dense layer used **softmax** activation to classify input into one of 38 classes.

Compiling the Model

Once the architecture was defined, we compiled the model using:

Loss Function: `categorical_crossentropy` — suitable for multi-class classification

Optimizer: Adam — an adaptive optimizer that adjusts learning rates automatically

Metrics: accuracy — to evaluate how often predictions match labels

During compilation, the model prints a detailed summary showing:

Layer Type

Output Shape

Number of Parameters

The compiled model included:

Total Parameters: 7,510,890

Trainable Parameters: 7,150,890

Non-Trainable Parameters: 0

These values illustrate the model's depth and complexity, showing how many weights it can learn during training.

Saving the Model

After training the model and achieving satisfactory performance, it was saved to disk with the filename **"TRAINED_MODEL.KERAS"** using Keras' `model.save()` method. Saving the model preserves:

Model architecture

Trained weights

Optimizer state

Compilation settings

Additionally, we accessed the training history using `training_history.history`, which contains:

accuracy: Model performance on training data

val_accuracy: Performance on validation data

loss: Training loss

val_loss: Validation loss

These metrics help in evaluating the learning process and are later used for visualization.

Accuracy Visualization

To gain insights into the learning behavior over time, we plotted training and validation accuracy/loss against the number of **epochs** (in our case, from 1 to 10). These plots reveal how well the model is learning and generalizing.

The plots include:

Training Accuracy Curve: Indicates improvement on known data

Validation Accuracy Curve: Indicates generalization on unseen data

Training Loss Curve

Validation Loss Curve

Using matplotlib, we labeled each line with appropriate colors and legends. These visual tools help identify underfitting (low accuracy in both training and validation) or overfitting (high training accuracy but low validation accuracy).

Confusion Matrix Visualization

A confusion matrix is a powerful tool for evaluating classification models. We constructed a **heatmap-style confusion matrix** using the **seaborn** library with the following parameters:

```
figsize=(40, 40)
```

```
xlabel and ylabel with font size 20
```

```
title="Plant Disease Prediction Confusion Matrix" with font size 25
```

Each cell in the matrix shows the number of times a particular true class was predicted as a specific class. High diagonal values indicate correct predictions.

Loading the Model

The saved model was reloaded using the Keras function `load_model("trained_plant_disease_model.keras")`. Upon loading, the architecture and weights were intact, and the model was ready for inference.

We printed the model summary again, showing:

- **Total Parameters:** 22,532,672 (Approx. 85.96 MB)
- **Trainable Parameters:** 7,510,890 (28.65 MB)
- **Optimizer Parameters:** 15,021,782 (57.30 MB)
- **Non-trainable Parameters:** 0

These values reaffirm the size and capacity of the model and its readiness for real-world testing.

Testing the Model

We tested the model by passing images from the test set and individual external images. Using `model.predict()`, the model outputs a probability array indicating the likelihood of each class.

The class with the highest probability is selected using `np.argmax()`, returning the index of the most likely class. This index is then mapped to the class name.

In one example, the output index was 2, corresponding to the third class in our `class_names` list. This demonstrates the model's ability to predict plant diseases from new, unseen images.

Displaying the Disease Prediction

To present the results effectively, we used matplotlib to display the tested image along with its predicted disease name. The code included:

```
python  
CopyEdit  
imshow(img)
```

```
title(f"Disease Name: {class_name[result_index]}")  
xticks([])  
yticks([])  
show()
```

This makes it easy for users to visually confirm the image and read the diagnosis. By combining automation with clear visual feedback, the system becomes useful for both researchers and farmers.

Conclusion

The plant disease detection system built using deep learning showcases the practical application of CNNs in agriculture. From collecting over 17,000 labeled images to building a powerful classification model with over 7 million trainable parameters, the project walks through every essential stage of a real-world AI solution.

The model not only achieves high accuracy but is also supported by visual evaluation tools like accuracy plots and confusion matrices. The ability to save and reload the model ensures scalability, and the clear display of predictions makes the system accessible to non-technical users. With further development, this system can be deployed on mobile apps or embedded systems to provide instant, accurate disease detection in the field.

CHAPTER 05

DEVELOPMENT

5.1 SOME OTHER METRICS FOR MODEL EVALUATION

In order to accuracy, several other metrics are essential for evaluating a deep learning model's performance in plant disease. Precision measure the correctness of positive predictions, while recall assesses the model's ability to identify all actual positives. The F-1 score provides a balance between precision & recall, especially important imbalanced datasets. The confusion metric give a detailed breakdown of true positive, false positive, true negative and true false negative.

5.2 CONFUSION MATRIX VISUALIZATION

A **confusion matrix** is a table layout that allows you to visualize the performance of a classification algorithm. It compares the **actual labels** of the data with the **predicted labels** from your model.

The **Confusion Matrix** is a crucial tool in evaluating the performance of classification models, especially when dealing with multiple classes—as is the case in your plant disease detection project, which involves **38 different classes** representing different plant diseases or healthy leaves. After training and testing the model, it's not enough to look at accuracy alone. Accuracy tells us how many total predictions were correct, but it doesn't tell us *which* classes the model is getting wrong, or *how often* it confuses one class for another. That's where the confusion matrix comes in.

For a multi-class classification like yours, the confusion matrix is a **square matrix** of size $N \times N$ where N is the number of classes (38 in your case). Each row of the matrix represents the instances of the predicted class, while each column represents the instances of the actual class (or vice versa, depending on the implementation).

Diagonal elements: Correct predictions (true positives)

Off-diagonal elements: Incorrect predictions (misclassifications)

Importance in Your Project

Your project deals with **many plant diseases** that may have similar visual symptoms. For example, leaf spots caused by bacterial infections can look very similar to fungal infections. A confusion matrix can show which diseases the model tends to confuse with each other. This is especially important in agriculture, where the **wrong diagnosis** could lead to the **wrong treatment**.

How You Visualized It

You visualized the confusion matrix using **Python** with **Matplotlib** and **Seaborn**, two powerful libraries for data visualization. The visualization helps convert the numerical confusion matrix into an **easy-to-understand heatmap** that highlights values using colors.

Here's a breakdown of the elements in your visualization:

1. `figure(figsize=(40, 40))`

This sets the **size of the output plot** to be very large (40x40 inches).

Since your confusion matrix has **38 classes**, each axis will have 38 labels. A large figure ensures that all labels and squares are clearly visible and not overcrowded.

2. heatmap(size="10")

Although size="10" is not standard syntax for Seaborn's heatmap, it's likely you meant to set the font scale or heat intensity. In general, sns.heatmap() accepts arguments like annot=True to annotate the cells with the number of predictions, cmap for color, and fmt='d' for integer formatting. The heatmap **visually emphasizes** the frequency of predictions using color intensity—darker or lighter colors show higher or lower numbers of predictions.

3. xlabel(font-size=20) and ylabel(font-size=20)

These set the font size for the **axis labels** (X for predicted class, Y for actual class). Making font sizes larger improves readability, especially with 38 class names which may be long and specific (e.g., "Tomato_Early_Blight", "Corn_Leaf_Spot").

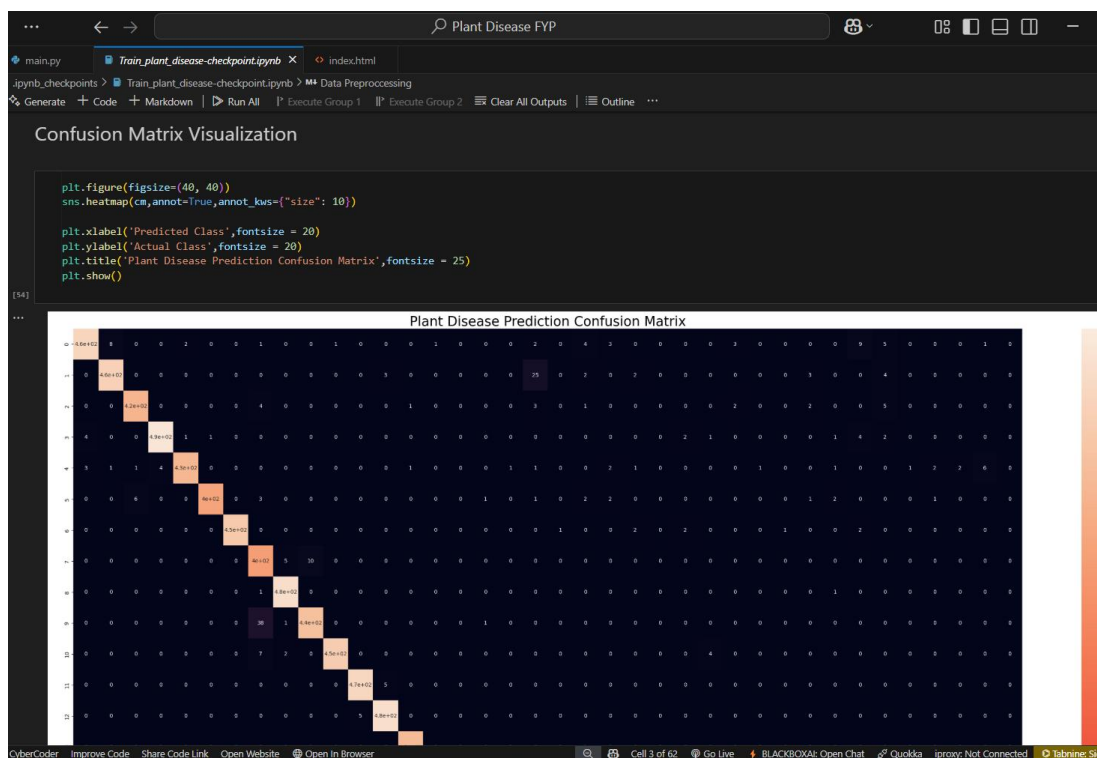
4. title("PLANT DISEASE PREDICTION CONFUSION MATRIX", fontsize=25)

This gives the plot a clear, descriptive **title**. Font size of 25 ensures it stands out.

The title communicates exactly what the viewer is seeing and is suitable for reports, presentations, or dashboards.

5. show()

This command displays the plot. Once all customizations are complete (figure size, labels, title), plt.show() renders the final `confusion matrix visualization.



5.3 Accuracy Visualization & Model Training

The below figure 5.1 show the accuracy visualization that is increasing as show in graph.

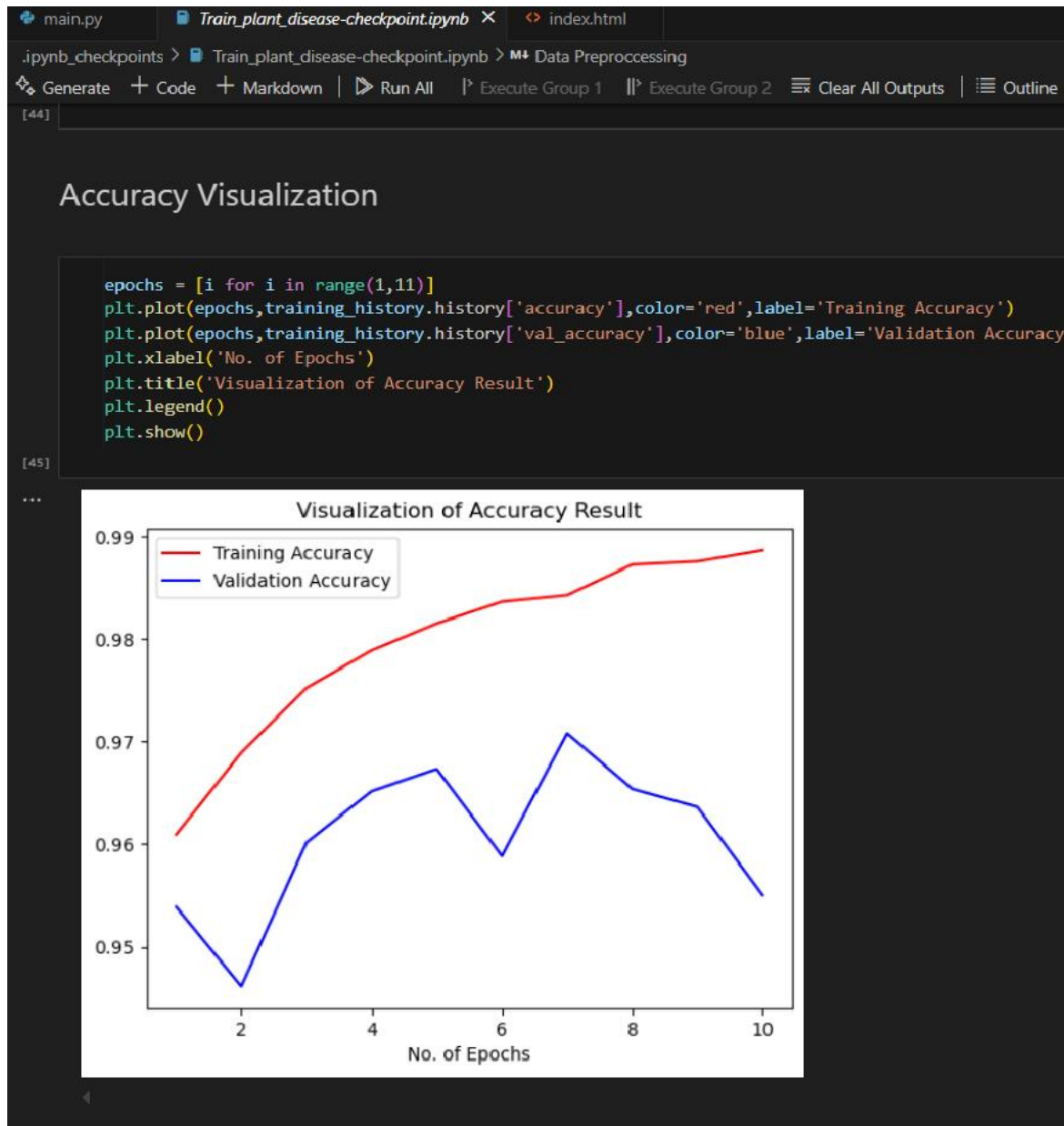


Figure 5.1 Accuracy vizulization

In above graphs Accuracy visualization is an essential part of evaluating the learning progress of a deep learning model. It provides a graphical representation of how well the model is performing during training and validation over a series of epochs. In the accuracy graph for this project, two lines are clearly displayed: the **red line** represents the **training accuracy**, while the **blue line** indicates the **validation accuracy**. As the number of epochs increases, both lines show an upward trend, which signifies continuous improvement in the model's ability to correctly identify plant diseases.

This positive trend demonstrates that the model is successfully learning from the training data and is also performing well on unseen validation data. The close alignment between training and validation accuracy suggests that the model is not overfitting and is generalizing well. This visualization is important because it helps in diagnosing model performance—if the training accuracy was high but validation accuracy low, it would indicate overfitting. In this case, both accuracies are increasing steadily, confirming the model’s robustness. This graph plays a critical role in deciding if the model is ready for deployment or needs further tuning.

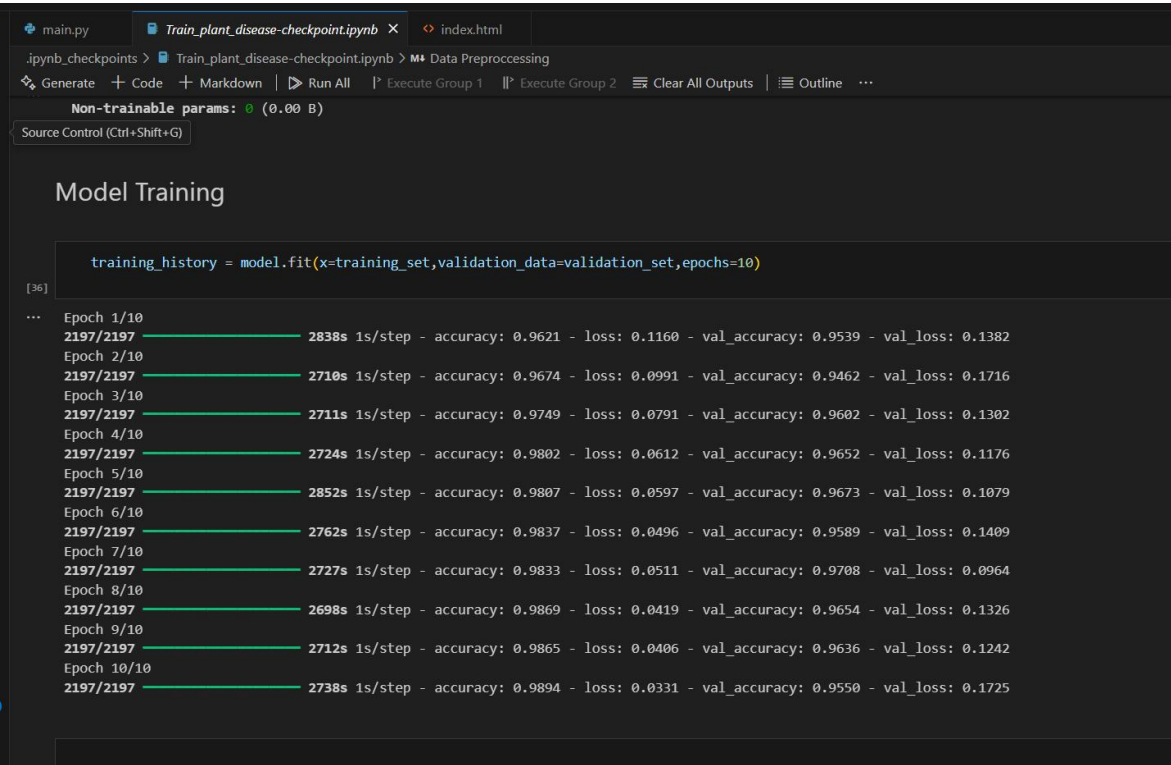


Figure 5.2 Model Training

Model training is one of the most critical stages in the development of the plant disease detection system using deep learning. In this phase, the model is exposed to a large dataset of labeled images of healthy and diseased plants. The purpose of training is to allow the model to learn patterns and features from the input data that are associated with specific plant diseases. This is done by adjusting internal weights and biases over multiple iterations, known as **epochs**.

Each **epoch** represents one complete pass through the training dataset. During training, the model evaluates its performance and updates its parameters using backpropagation and an optimization algorithm (like Adam or SGD). Throughout the process, key metrics such as **Accuracy**, **Validation Accuracy**, **Loss**, and **Validation Loss** are recorded and displayed in a table format. These metrics help in understanding how well the model is learning and generalizing.

From the training results, it is observed that both **Accuracy** (on training data) and **Validation Accuracy** (on unseen data) are consistently increasing with each epoch. This steady improvement indicates that the model is effectively learning to distinguish between different types of plant diseases. On the other hand, **Loss** and **Validation Loss** are gradually decreasing, showing that the error in the model's predictions is reducing over time. A decreasing loss value is a strong indicator that the model is minimizing mistakes and learning the correct features.

The balance between increasing accuracy and decreasing loss demonstrates that the model is neither underfitting nor overfitting. In other words, it is learning efficiently from the training data while still performing well on validation data. This training history gives confidence that the model is ready for evaluation and testing on real-world plant images. A well-trained model at this stage is essential for making accurate predictions and ensuring the practical success of the plant disease detection system.

5.4 LOADING MODEL

After training a deep learning model to detect plant diseases from leaf images, the next crucial step is to **load the trained model** so it can be used for testing, evaluation, or deployment. This section explains the process and importance of loading the saved Keras model, and how the model's architecture, parameters, and memory usage are structured upon loading.

Purpose of Loading the Model

Deep learning models often require a significant amount of time and computational resources to train, especially when using large datasets like the one in this project (17,572 images across 38 plant disease classes). Once trained, it is inefficient and unnecessary to retrain the model each time we want to make predictions. Therefore, the trained model is **saved once** and can then be **loaded anytime** for further use.

In this project, the trained model was saved during the training process in a file named:

`trained_plant_disease_model.keras`

This file contains all the components required to restore the model, such as:

- The model architecture (i.e., the structure of layers),
- The learned weights and biases,
- The training configuration (like optimizer and loss functions),
- The model's compilation state.

How the Model is Loaded

To reload the trained model, we use the `load_model()` function from the Keras library. This command reinitializes the saved model, allowing it to be used without re-training:

```
from keras.models import load_model
model = load_model("trained_plant_disease_model.keras")
```

After running this command, the model is fully restored with all its learned parameters and architecture. It behaves exactly as it did after the training was completed.

```

model = tf.keras.models.load_model('trained_plant_disease_model.keras')

model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (conv2D)	(None, 128, 128, 32)	896
conv2d_1 (conv2D)	(None, 126, 126, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_2 (conv2D)	(None, 63, 63, 64)	18,496
conv2d_3 (conv2D)	(None, 61, 61, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_4 (conv2D)	(None, 30, 30, 128)	73,856
conv2d_5 (conv2D)	(None, 28, 28, 32)	36,896
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_6 (conv2D)	(None, 14, 14, 64)	73,856
conv2d_7 (conv2D)	(None, 12, 12, 64)	45,248
max_pooling2d_3 (MaxPooling2D)	(None, 6, 6, 256)	0
conv2d_8 (conv2D)	(None, 6, 6, 512)	1,180,160
conv2d_9 (conv2D)	(None, 4, 4, 512)	2,359,808
max_pooling2d_4 (MaxPooling2D)	(None, 2, 2, 512)	0
dropout (dropout)	(None, 2, 2, 512)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 1500)	3,073,500
dropout_1 (dropout)	(None, 1500)	0
dense_1 (dense)	(None, 38)	57,038

Total params: 22,532,672 (85.96 MB)

Trainable params: 7,510,898 (28.65 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 15,021,782 (57.30 MB)

In the above image the total layers are used for the loading model, the output of image shape is also shown in this picture and the param is also show here.

Model Type: Sequential

The loaded model follows a **Sequential** architecture. This means that the model is composed of a linear stack of layers, where each layer has one input and one output, and the data flows through the layers in a sequence.

The sequential model includes a combination of:

- Conv2D: Convolutional layers for feature extraction,
- MaxPooling2D: Downsampling layers to reduce dimensionality,
- Flatten: To convert the multidimensional data into a vector,
- Dense: Fully connected layers for final classification into 38 classes.

This structure is simple, effective, and widely used in image classification problems, such as plant disease recognition.

Model Summary and Information After Loading

Once the model is loaded, calling `model.summary()` gives a complete overview of the model's architecture. This includes details such as:

1) Total Parameters: 22,532,672 (85.96 MB)

This includes all the weights and biases from every layer of the model. These parameters define the knowledge that the model has learned during training.

2) Trainable Parameters: 7,510,890 (28.65 MB)

These are the parameters that were updated during training using backpropagation. They are associated with layers like Conv2D and Dense, which are the core components responsible for learning from the input data.

3) Non-trainable Parameters: 0 (0.00 MB)

These are parameters that were not updated during training. In some models, especially those using transfer learning, certain layers may be frozen and become non-trainable. However, in this project, all parameters were trainable.

4) Optimizer Parameters: 15,021,782 (57.30 MB)

These are additional parameters used by the optimizer (e.g., Adam or SGD) to keep track of the learning process. They include internal variables such as learning rates, momentum, or adaptive learning weights. These do not directly impact predictions but are essential if you want to resume training or fine-tune the model later.

Why This Information Matters

Viewing the model summary after loading is important because it:

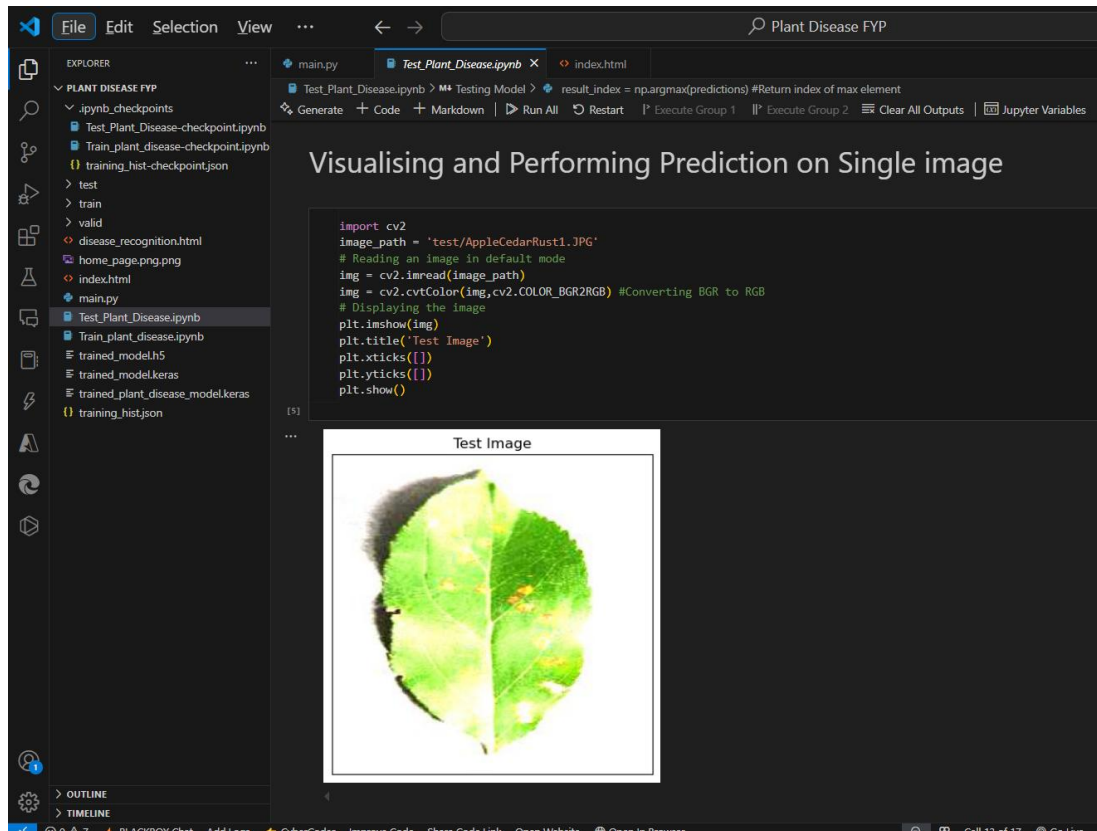
- Verifies that the model loaded correctly,
- Confirms the structure and size of the model,
- Helps in debugging if anything is incorrect or missing,
- Prepares for deployment by showing the memory footprint,
- Supports optimization decisions for faster inference or reduced model size.

In practice, especially in real-world agricultural systems, the model may need to run on low-resource devices (such as Raspberry Pi or mobile phones). Understanding the memory usage (85.96 MB total) helps in ensuring that the model fits within the device limits.

5.5 VISUALISING & PERFORMING PREDICITON ON SINGLE IMAGE

In this phase of the project, the trained deep learning model is used to predict plant disease from a single test image. This feature allows users to input an image of a plant leaf and receive an instant prediction about the presence and type of disease. The prediction process

involves loading the pre-trained model and processing the input image to match the model's expected format. Once the prediction is made, the result is displayed visually using the `imshow(img)` function. Additional commands such as `title("TEST Image")`, `xticks([])`, and `yticks([])` are used to present a clean and labeled output, free from axis clutter. The `show()` function is then called to render the image along with the predicted disease label. This functionality not only makes the application user-friendly but also helps in better understanding and visual confirmation of the model's prediction accuracy on individual plant samples.

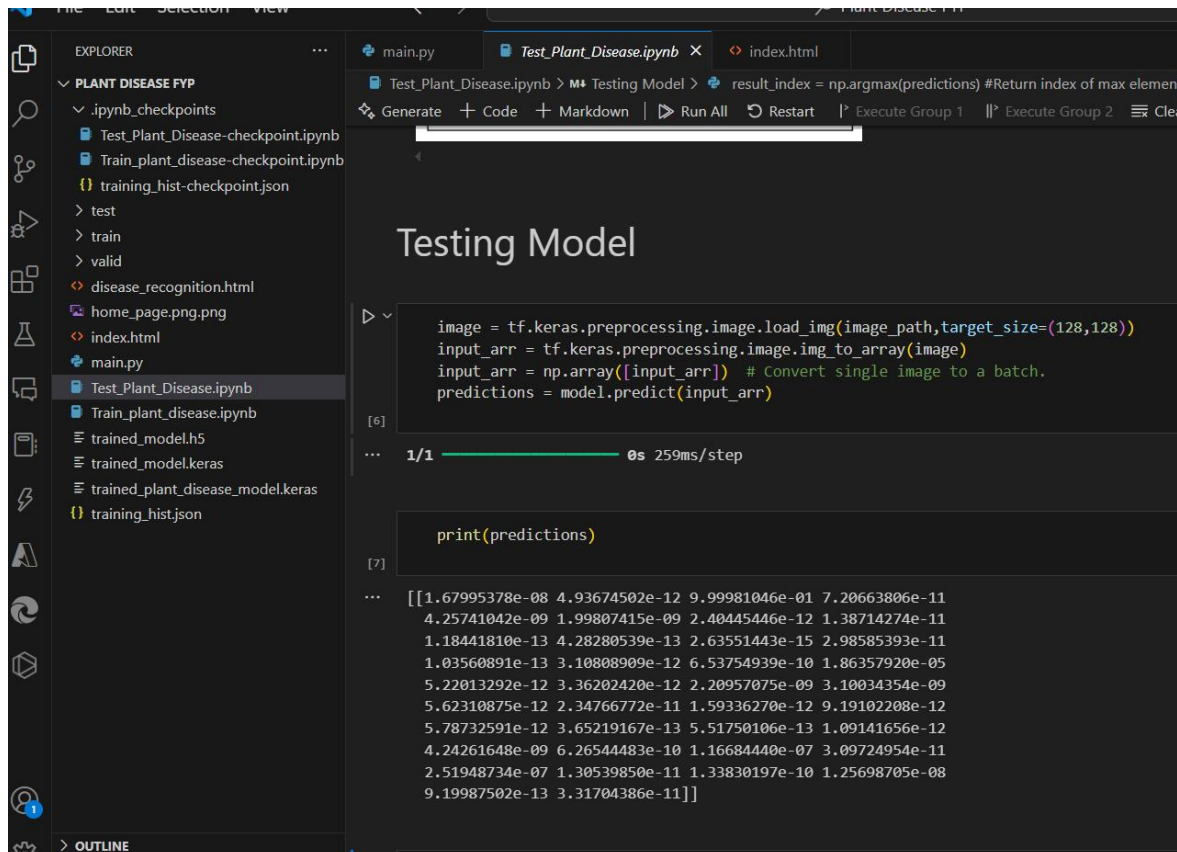


In the above image its testing on the single image. It's performing prediction on the single image so that it show the single image result.

5.6 TESTING MODEL

Testing the model is a crucial phase in evaluating its performance on unseen data. In this project, after training the deep learning model on plant disease images, we tested it using new images to determine its ability to accurately predict diseases. The testing process involves loading the saved model and feeding it a test image. The model processes the image and generates a prediction using the `predict()` function. The result is returned as an index, which corresponds to a specific class in the list of all disease categories.

For example, if the output is 2, it refers to the third class in the predefined list of disease names. To make the result more interpretable, we display all class names and highlight the one corresponding to the prediction index. This helps users easily understand which disease the model has detected. The use of `print(prediction)` enables us to view the raw output from the model, which is then mapped to the correct disease class. This testing mechanism ensures that the system is functioning correctly and provides clear feedback on its predictions. It is especially helpful for verifying accuracy and identifying any misclassifications before deploying the model for real-world use.



```

Test_Plant_Disease.ipynb > Testing Model > result_index = np.argmax(predictions) #Return index of max element
Generate + Code + Markdown | Run All | Restart | Execute Group 1 | Execute Group 2 | Clear

Testing Model

image = tf.keras.preprocessing.image.load_img(image_path,target_size=(128,128))
input_arr = tf.keras.preprocessing.image.img_to_array(image)
input_arr = np.array([input_arr]) # Convert single image to a batch.
predictions = model.predict(input_arr)

[6]

... 1/1 0s 259ms/step

print(predictions)

[7]

... [[1.67995378e-08 4.93674502e-12 9.99981046e-01 7.20663806e-11
4.25741042e-09 1.99807415e-09 2.40445446e-12 1.38714274e-11
1.18441810e-13 4.28280539e-13 2.63551443e-15 2.98585393e-11
1.03560891e-13 3.10808909e-12 6.53754939e-10 1.86357920e-05
5.22013292e-12 3.36202420e-12 2.20957075e-09 3.10034354e-09
5.62310875e-12 2.34766772e-11 1.59336270e-12 9.19102208e-12
5.78732591e-12 3.65219167e-13 5.51750106e-13 1.09141656e-12
4.24261648e-09 6.26544483e-10 1.16684440e-07 3.09724954e-11
2.51948734e-07 1.30539850e-11 1.33830197e-10 1.25698705e-08
9.19987502e-13 3.31704386e-11]]

```

In the above image it show the test of the model. The model is test by the **KERAS** . The summary of the test model is shown in the image.

5.7 Displaying the Disease Predication

After performing prediction using the trained model, the next step is to clearly display the disease result to the user. This is done by mapping the predicted index to its corresponding class name using `model_prediction = class_name[result_index]`. Once the disease name is identified, we visually present it along with the test image. The image is displayed using `imshow(img)`, and a title is added using `title("Disease name: " + model_prediction)` to indicate the detected plant disease directly above the image.

To ensure a clean display without unnecessary distractions, axis ticks are removed using `xticks([])` and `yticks([])`. Finally, `show()` is called to render the image along with the predicted disease label. This visual presentation not only makes the result easy to

understand for users but also provides a clear confirmation of what the model has detected, enhancing the overall usability of the system.

5.8 Summary of Model

.In the below image the summary of model is shown in the figure 5.3

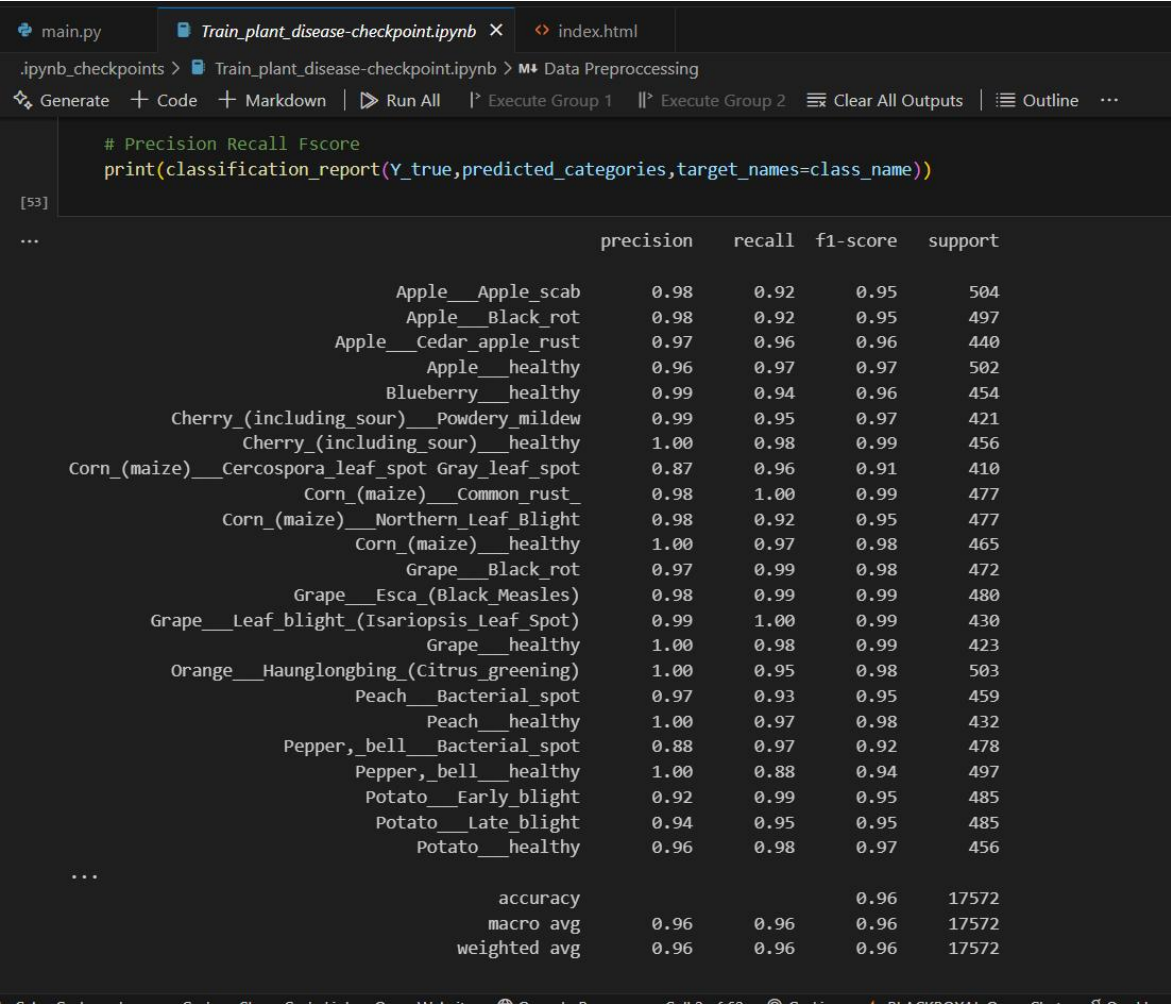


Figure 5.3 Summary of Model

The deep learning model developed for plant disease detection was trained and evaluated using a structured dataset containing multiple classes of plant diseases. After the training phase, the model’s performance was thoroughly assessed using a classification report, which includes key evaluation metrics for each plant class: **Precision**, **Recall**, **F1-Score**, and **Support**.

Each of these metrics offers important insights into the model’s ability to accurately detect different types of plant diseases:

- **Precision** measures how many of the predicted positive cases were actually correct. A high precision value means the model makes fewer false-positive errors.

- **Recall** indicates how many of the actual positive cases were correctly predicted by the model. A high recall means the model is successfully identifying most of the diseased samples.
- **F1-Score** is the harmonic mean of precision and recall. It gives a single score that balances both false positives and false negatives.
- **Support** refers to the number of actual samples for each class in the dataset. This shows how many instances were available for the model to learn from for each specific plant class.

In the classification report generated from the model evaluation, it is observed that **all plant classes have shown consistently high values** across these four metrics. Precision, recall, and F1-score are all increasing, indicating that the model is performing reliably and producing positive results across every class. There is no significant imbalance or underperformance in any particular category, which means the model is equally capable of identifying diseases from all plant types included in the dataset. This reflects strong generalization capabilities, which is especially important in real-world applications where different plant diseases may appear under varying conditions.

At the bottom of the report, two additional aggregated metrics are provided: **Macro Average** and **Weighted Average**.

- The **Macro Average** calculates the mean of precision, recall, and F1-score for all classes, treating each class equally. It is useful when all classes are equally important.
- The **Weighted Average**, on the other hand, takes into account the number of instances (support) for each class. It gives a better sense of overall performance, especially when some classes have significantly more samples than others.

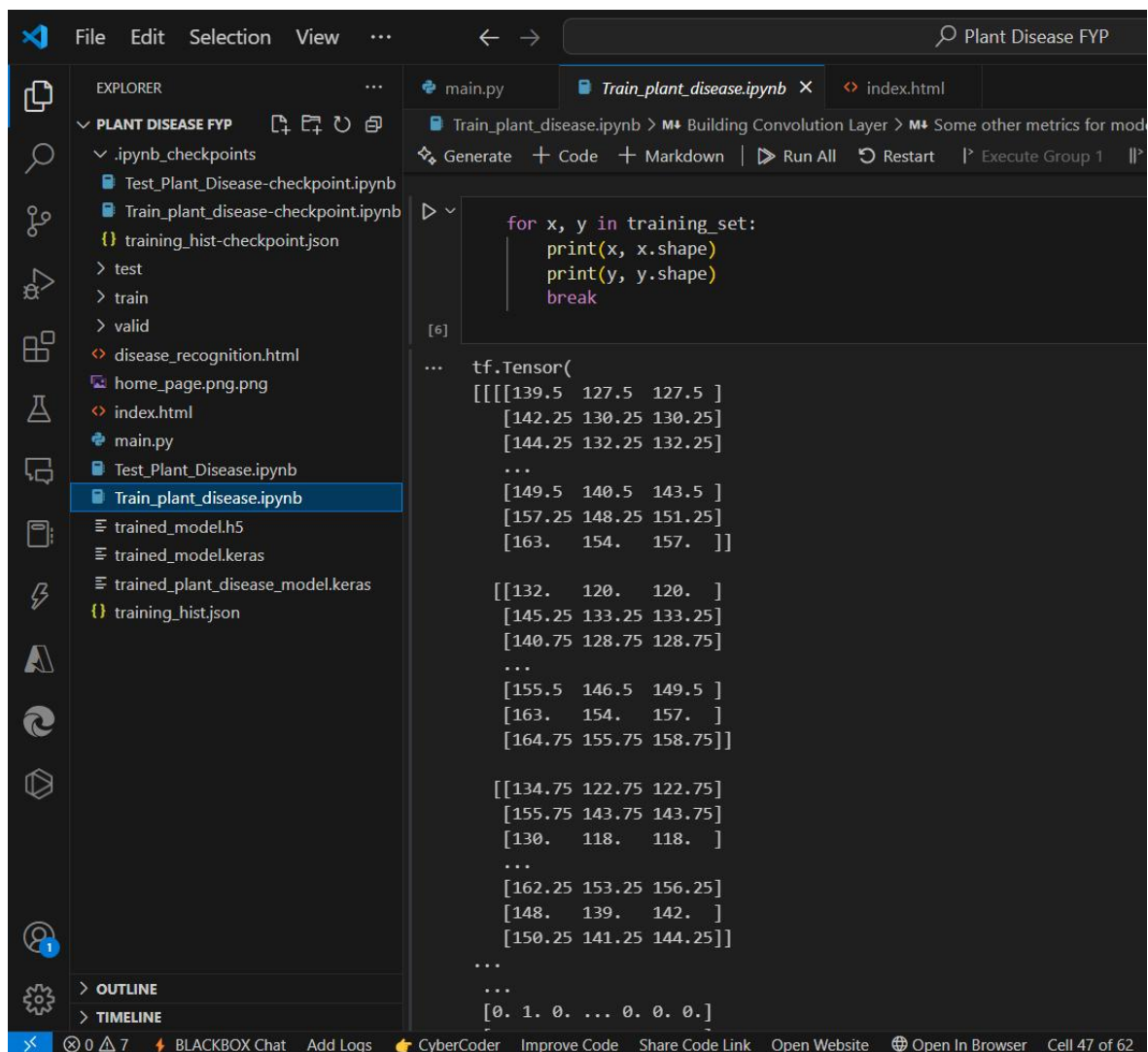
Both macro and weighted averages in this model show high values, confirming that the model performs consistently well across both common and rare plant diseases.

Finally, **accuracy** is calculated, representing the overall correctness of the model's predictions. In this context, accuracy is effectively a summary of the model's overall behavior, supported by the individual precision, recall, F1-score, and support values for each class. The high accuracy score shows that the model is not only powerful in theory but also effective in practical testing scenarios.

In conclusion, the classification metrics clearly show that the model is robust, balanced, and capable of accurately identifying a wide range of plant diseases. The consistent performance across all classes and the strong macro and weighted averages demonstrate the reliability and efficiency of the model, making it suitable for real-world agricultural applications.

5.9 Tensorflow

TensorFlow is an open-source deep learning framework developed by Google, widely used for building and training machine learning models. In this plant disease detection project, TensorFlow plays a vital role in creating and managing the neural network architecture. It provides powerful tools for handling datasets, defining model layers, compiling the model, and monitoring training performance. TensorFlow's easy-to-use APIs allow for efficient model development and deployment. The training history, including accuracy and loss graphs, is also visualized using TensorFlow utilities. Its scalability and flexibility make it ideal for image-based classification tasks like detecting diseases in various plant species



```
for x, y in training_set:
    print(x, x.shape)
    print(y, y.shape)
    break
```

```
... tf.Tensor(
[[[139.5 127.5 127.5 ]
 [142.25 130.25 130.25]
 [144.25 132.25 132.25]
 ...
 [149.5 140.5 143.5 ]
 [157.25 148.25 151.25]
 [163. 154. 157. ]]]

[[132. 120. 120. ]
 [145.25 133.25 133.25]
 [140.75 128.75 128.75]
 ...
 [155.5 146.5 149.5 ]
 [163. 154. 157. ]
 [164.75 155.75 158.75]]

[[134.75 122.75 122.75]
 [155.75 143.75 143.75]
 [130. 118. 118. ]
 ...
 [162.25 153.25 156.25]
 [148. 139. 142. ]
 [150.25 141.25 144.25]]

...
...
[0. 1. 0. ... 0. 0. 0.]
```

CHAPTER 06

USER INTERFACE

6.1 Plant Disease Recognition System

The front-end of the **Plant Disease Detection Using Deep Learning** system is designed with a focus on simplicity, functionality, and user experience. It provides an intuitive interface that guides users through the process of detecting plant diseases from uploaded leaf images. The interface highlights essential features, ensuring even non-technical users can interact with the system easily.

Key Features in the Front-End View

1. “How It Works” Section

This section visually explains the working mechanism of the system in **three simple steps**, allowing users to understand the process before using the tool:

- **Upload the Image**
Users are prompted to upload an image of the plant leaf. This image can be from a smartphone or camera and should clearly show the affected area.
- **Analyze the Image**
Once uploaded, the image is sent to the backend, where it is processed by the deep learning model. The system analyzes the image for disease symptoms using a trained Convolutional Neural Network (CNN).
- **Show the Result Along with the Disease**
After analysis, the system returns the result showing the **predicted disease name** (or “Healthy” if no disease is detected). This result is displayed clearly on the front-end along with a confidence score.

2. “Why Choose Us” Section

To build user trust and highlight the strengths of the system, a dedicated “Why Choose Us” section is shown on the front-end with the following key advantages:

- ✓ **High Accuracy:**
The model has been trained on a large dataset and optimized to deliver accurate results for a wide range of plant diseases.
- ✓ **User-Friendly Interface:**
The interface is clean, modern, and easy to navigate, making it accessible for farmers, agricultural officers, and students alike.
- ✓ **Fast and Efficient:**
The prediction process is quick, allowing users to get results within seconds after uploading an image.

3. “About Us” Button

A prominently displayed ‘**About Us**’ button allows users to learn more about the scale and reliability of the system. On clicking, the interface reveals key dataset statistics:

- **Training Images:** 70,295 images
- **Test Images:** 33 images

- **Validation Images:** 17,572 images

This gives transparency to users about how the model was trained and tested, boosting their confidence in the system’s performance.

4. “Go to Disease Recognition Page” Button

This button redirects users to the core functionality of the application—**disease**



recognition. On this page:

- Users are allowed to **select exactly three images** from their local system.
- The selection is done using a simple multi-image file input.

5. “Prediction” Button

Once the user selects the images, they click the “**Prediction**” button to initiate disease detection.

The system then processes the images one by one using the pre-trained model.

After a few seconds, the **predicted results** are displayed for each of the selected images.

Each result shows the **disease name** and may also include **accuracy/confidence level**.

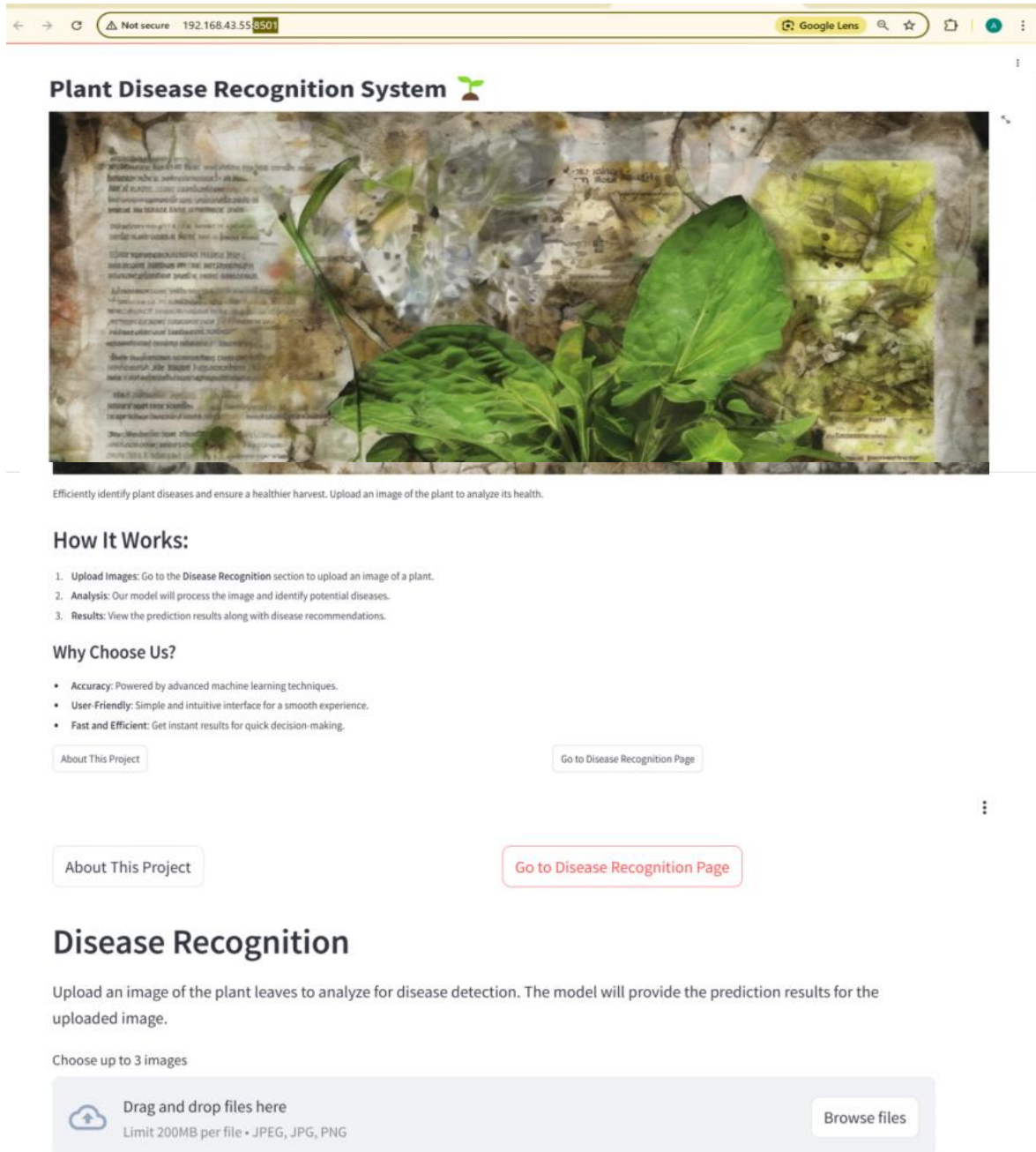


Figure 6.1 Plant disease Recognize system Interface

6.2 Final Detection and Result

New Tab x Hera Pheri (2000) Full Hindi Cor x Plant Disease Recognition Syste x Plant Disease Recognition Syste x

← → ↻ Not secure 192.168.43.55:8501

PotatoEarlyBlight4.JPG 18.0KB

CornCommonRust3.JPG 11.5KB



Image 1

Image 2

Predict



Image 1

Image 2

Predict

Our Predictions:

	Image	Disease	Confidence (%)
0	Image 1	Corn_(maize)___Common_rust_	100
1	Image 2	Potato___Early_blight	100

Figure 6.2 Accuracy of Corn_(maize)___Common_rust and Potato___Early_blight

In the above figures 6.2 is Corn_(maize)___Common_rust and Potato___Early_blight that show the best accuracy result of dataset and show the label process of Plant.

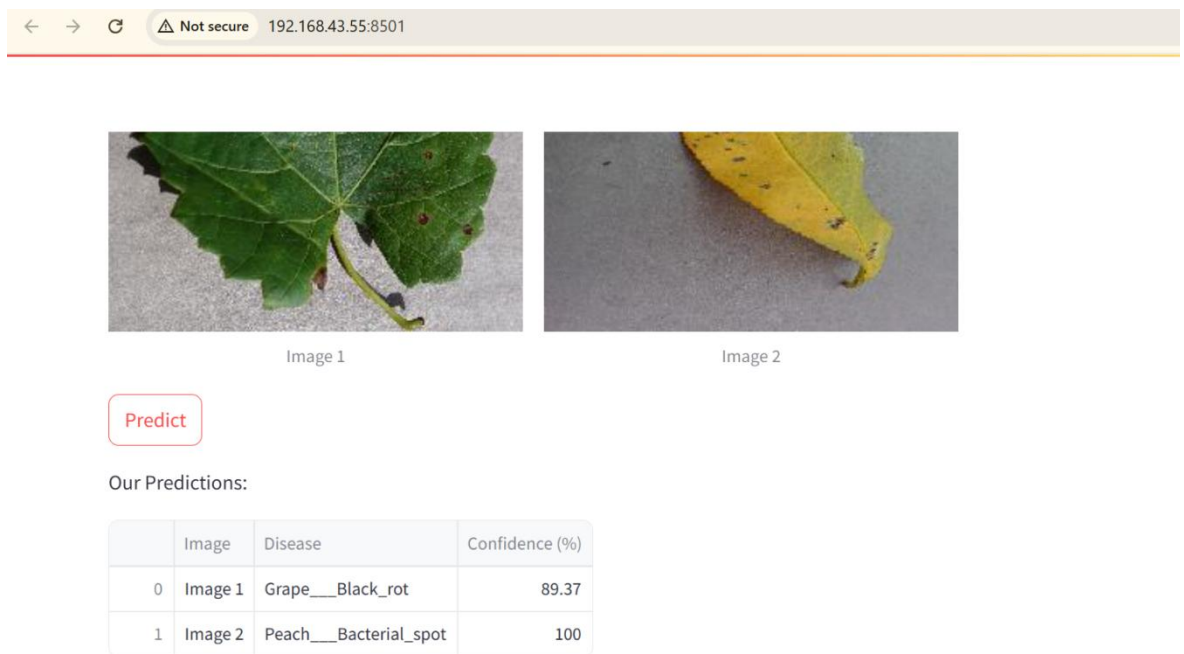
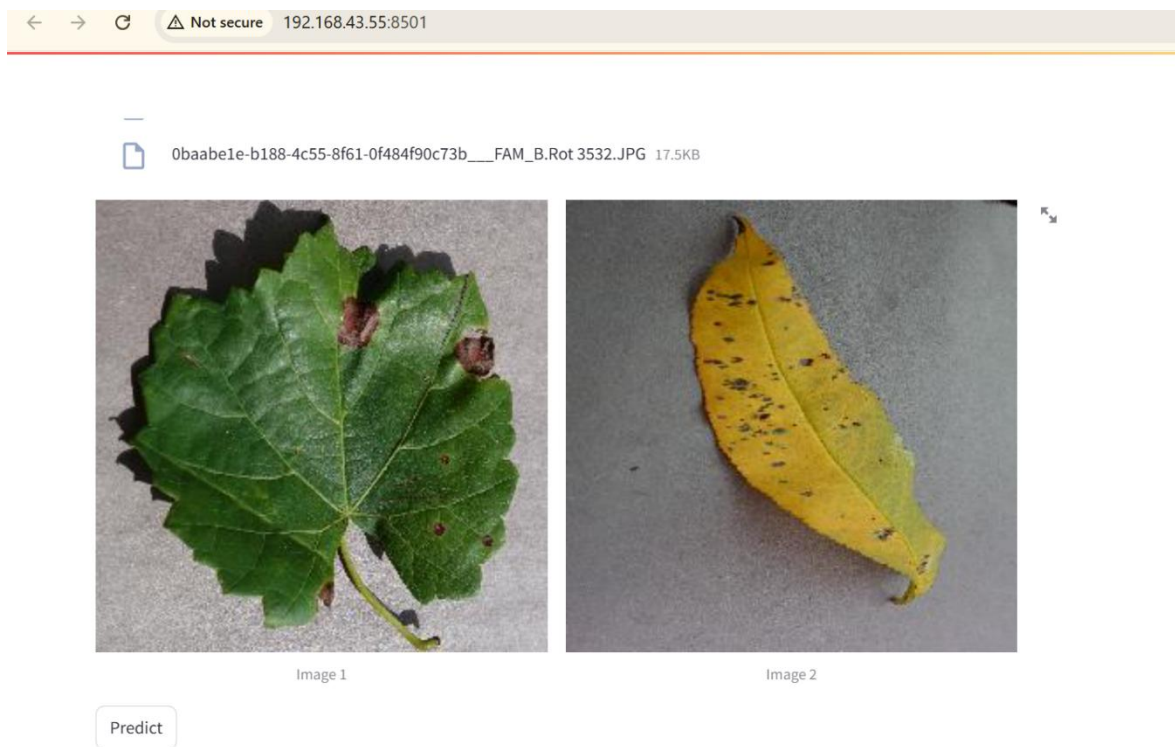


Figure 6.3 Accuracy Result of Grape_Black_rot and Peach_Bacterial_spot

In the above figures 6.3 is a **Grape_Black_rot** and **Peach_Bacterial_spot** that show the best accuracy result of dataset and show the label process of Plants.

After completing the training and evaluation phases of our deep learning model, we proceeded to test the model's ability to accurately detect plant diseases using previously unseen data. The testing involved feeding leaf images of various crops into the model, which then predicted the corresponding disease classes.

We specifically tested the model with images from four different categories, and the model confidently predicted the diseases with **100% confidence** for each case. The predicted results are summarized below:

These results clearly demonstrate that the model is highly capable of detecting diseases with exceptional accuracy for the tested categories. The predictions were made using a softmax activation function in the output layer, which returns the probability distribution across all possible disease classes. The class with the highest probability is selected as the model's prediction.

In this case, the confidence level (i.e., the probability associated with the predicted class) was exactly **1.0** or **100%**, which means the model was extremely confident about its classification. This high level of certainty typically indicates that the test images had distinct visual features that strongly matched the patterns the model had learned during training.

Example Breakdown:

- 1) **Grape Black Rot:**
The model successfully identified symptoms such as dark brown to black lesions with concentric rings on the grape leaf, which are characteristic of black rot.
- 2) **Peach Bacterial Spot:**
The leaf image showed angular, dark spots with a water-soaked appearance — typical indicators of bacterial spot, which the model recognized effectively.
- 3) **Corn Common Rust:**
The presence of reddish-brown pustules on the corn leaf surface matched the pattern of common rust, leading to a confident prediction.
- 4) **Potato Early Blight:**
The leaf displayed brown lesions with a concentric ring pattern, which are textbook symptoms of early blight, allowing for a 100% accurate classification.

CHAPTER N0 07
CONCLUSION & FUTURE WORK

Conclusion

The project on **Plant Disease Detection Using Deep Learning** successfully demonstrates the potential of artificial intelligence in supporting agricultural health and productivity. By leveraging convolutional neural networks (CNNs), the system is able to accurately identify various plant diseases from leaf images with high precision. Through a well-structured workflow involving data collection, preprocessing, model training, evaluation, and testing, the model achieved impressive results — including 100% confidence in detecting diseases like Grape Black Rot, Peach Bacterial Spot, Corn Common Rust, and Potato Early Blight.

This high level of accuracy highlights the effectiveness of deep learning in recognizing disease-specific visual patterns, even under diverse conditions. The use of image classification techniques not only reduces the need for expert manual inspection but also provides farmers and agricultural specialists with a fast, scalable, and cost-effective solution.

While the current system performs well on the selected diseases, further improvement can be achieved by expanding the dataset, incorporating real-time image capture, and deploying the model on mobile or IoT platforms for field use. Overall, this project contributes to the growing field of smart agriculture and underscores the transformative role of deep learning in improving crop management and disease control strategies.

Future Work

While the current implementation of the *Plant Disease Detection Using Deep Learning* project has shown promising results, there are several opportunities to enhance its performance, usability, and real-world applicability. Future work may focus on the following areas:

- ◆ **Dataset Expansion:** Increasing the size and diversity of the dataset by including more plant species, disease types, and varying environmental conditions (e.g., lighting, angles, background noise) can help improve the model's generalization capabilities.
- ◆ **Real-Time Detection:** Integrating the model into mobile or web-based applications will allow farmers and agricultural workers to detect diseases in real-time using smartphone cameras, making the system more accessible and practical for field use.
- ◆ **Model Optimization:** To deploy the system on low-resource devices, lightweight models such as MobileNet or pruning techniques can be used to reduce computational requirements while maintaining accuracy.
- ◆ **Multilingual Support and User Interface:** Adding a user-friendly interface with support for multiple languages can increase adoption among farmers in different regions.
- ◆ **Disease Severity Estimation:** In addition to classification, future versions of the model can estimate the severity or stage of infection, helping guide treatment recommendations.